PERSONAAI: AN ADAPTIVE AI STUDY ASSISTANT WITH RETRIEVAL-AUGMENTED GENERATION AND WEAK-TOPIC MEMORY

A Mini Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE

by

[Student Name 1] ([Roll No.])

[Student Name 2] ([Roll No.])

[Student Name 3] ([Roll No.])

[Student Name 4] ([Roll No.])

Under the guidance of

[Guide Name]

[Designation], Department of Artificial Intelligence

[INSTITUTE LOGO]

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

ANURAG UNIVERSITY

[Affiliation / Accreditation, City – PIN]

[Month, Year]

PERSONAAI: AN ADAPTIVE AI STUDY ASSISTANT WITH RETRIEVAL-AUGMENTED GENERATION AND WEAK-TOPIC MEMORY

A Mini Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE

by

[Student Name 1] ([Roll No.])

[Student Name 2] ([Roll No.])

[Student Name 3] ([Roll No.])

[Student Name 4] ([Roll No.])

Under the guidance of

[Guide Name]

[Designation], Department of Artificial Intelligence

[INSTITUTE LOGO]

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

ANURAG UNIVERSITY

[Affiliation / Accreditation, City – PIN]

[Month, Year]

ANURAG UNIVERSITY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

[INSTITUTE LOGO]

CERTIFICATE

This is to certify that the Mini Project report entitled **"PersonaAI: An Adaptive AI Study Assistant with Retrieval-Augmented Generation and Weak-Topic Memory"** is a bonafide record of the work carried out by [Student Name 1] ([Roll No.]), [Student Name 2] ([Roll No.]), [Student Name 3] ([Roll No.]), [Student Name 4] ([Roll No.]) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence during the academic year 2026–2027. The work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Project Guide	**Head of the Department**
[Guide Name]	[HOD Name]
[Designation]	Department of AI

External Examiner

DECLARATION

We hereby declare that the Mini Project report entitled **"PersonaAI: An Adaptive AI Study Assistant with Retrieval-Augmented Generation and Weak-Topic Memory"** submitted to the Department of Artificial Intelligence, Anurag University, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology, is a record of original work carried out by us under the guidance of [Guide Name, Designation]. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma. Wherever the work of others, including datasets, pretrained models and publicly available code, has been used, it has been duly acknowledged and cited. The results reported are genuine and reproducible from the submitted code.

Use of AI tools: We declare that we have used [name of tool(s) — state every tool actually used, e.g. an AI coding assistant] for [purpose, e.g. code debugging, language editing]. We take full responsibility for the content of this report.

Place: Hyderabad

Date: [DD-MM-YYYY]

Name of the Student	Roll No.	Signature
[Student Name 1]	[Roll No.]	
[Student Name 2]	[Roll No.]	
[Student Name 3]	[Roll No.]	
[Student Name 4]	[Roll No.]	

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide [Guide Name], [Designation], Department of Artificial Intelligence, for the valuable guidance, constant encouragement and support throughout this project.

We thank [HOD Name], Head of the Department of Artificial Intelligence, and the Project Review Committee members for their suggestions during the reviews.

We are grateful to [Dean SOE], the Vice Chancellor, and the management of Anurag University for providing the facilities required to complete this work.

We also thank all the faculty and staff of the department, our classmates who tried the application and shared feedback, and our families and friends for their support.

[Student Name 1] ([Roll No.])
[Student Name 2] ([Roll No.])
[Student Name 3] ([Roll No.])
[Student Name 4] ([Roll No.])

ABSTRACT

Students preparing for university examinations usually revise from scattered lecture notes and handouts. General-purpose artificial intelligence chatbots explain concepts fluently, but answer from general knowledge rather than the course material, cannot show where an answer came from, and do not remember which topics a student keeps getting wrong.

This project presents PersonaAI, a web-based study assistant built around the student's own notes. Uploaded files are split into page-bounded passages, encoded by a small local sentence-embedding model and stored in a vector database. Questions are answered by retrieval-augmented generation: twenty candidate passages are retrieved, a cross-encoder re-ranks them, and a large language model (Google Gemini, with a local Llama 3.1 model as an offline fallback) answers only from the best five, citing file and page, or states that the notes do not contain the answer. The same notes generate validated multiple-choice quizzes. A weak-topic memory tracks each topic's accuracy over the last ten answers; adaptive quizzes place about 60 per cent of questions on weak topics, and a rule-based planner builds a revision timetable that gives weak topics three times the time of strong ones.

The system was evaluated on 60 examination questions from a Big Data course, split into development and held-out halves. On the held-out half, notes-grounded answers were fully correct for 25 of 30 questions and partially correct for five, with none factually wrong, while the same model without notes answered all 30 correctly; the shortfall came from passages that retrieval missed or cut off. In a simulation of 200 students, adaptive quizzes placed 55 per cent of questions on genuinely weak topics, against 25 per cent for evenly spread quizzes.

Grounding answers in course notes therefore trades some completeness for verifiable answers, and passage context is the main direction for improvement.

Keywords: Retrieval-Augmented Generation, Large Language Models, Adaptive Learning, Learner Modelling, Cross-Encoder Re-ranking, Educational Technology

TABLE OF CONTENTS

Certificate i
Declaration ii
Acknowledgement iii
Abstract iv
List of Figures vii
List of Tables viii
List of Abbreviations ix
CHAPTER 1 INTRODUCTION 1
1.1 Background 1
1.2 Motivation 1
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Scope and Limitations 3
1.6 Organization of the Report 3
CHAPTER 2 LITERATURE SURVEY 4
2.1 Review of Relevant Work 4
2.2 Summary of the Literature 7
2.3 Research Gap Identified 9
CHAPTER 3 SYSTEM ANALYSIS AND REQUIREMENTS 10
3.1 Existing Approaches and their Drawbacks 10
3.2 Feasibility Study 10
3.3 Hardware and Compute Used 11
3.4 Software Requirements 11
3.5 Functional and Non-Functional Requirements 12
CHAPTER 4 DATASET, DESIGN AND METHODOLOGY 14
4.1 Overall Pipeline 14
4.2 Dataset 14
4.3 Pre-processing 15
4.4 Development / Held-out Split and Avoiding Data Leakage 16
4.5 System Architecture and Model Choice 17
4.6 Methodology 18
4.7 Baseline Models for Comparison 25
4.8 Comparison with Recent (State-of-the-Art) Models 25
CHAPTER 5 IMPLEMENTATION 26

5.1 Inference and Configuration Setup....26
5.2 Parameter Selection Approach....26
5.3 Key Code Snippets....27
5.4 Deployment and User Interface....33
5.5 Development Process and Testing....34
CHAPTER 6 RESULTS AND DISCUSSION....37
6.1 Evaluation Metrics....37
6.2 Training and Validation Behaviour....37
6.3 Test-Set (Held-Out) Results....38
6.4 Comparison with Baselines and Published Work....39
6.5 Error Analysis....41
6.6 Ablation Study....42
6.7 Output Screenshots....43
6.8 Ethical and Responsible AI....50
CHAPTER 7 CONCLUSION AND FUTURE SCOPE....52
7.1 Conclusion....52
7.2 Limitations....52
7.3 Future Enhancements....53
REFERENCES....54
APPENDICES....57
Appendix A: GitHub Repository and README....57
Appendix B: Model Links....57
Appendix C: Project Structure and Key Code....57
Appendix D: Plagiarism Report....58
Appendix E: Publications and Certificates....58

LIST OF FIGURES

Fig. No.	Title	Page No.
4.1	Overall pipeline of PersonaAI: data, pre-processing, models, evaluation and deployment	14
4.2	Document ingestion and pre-processing pipeline	16
4.3	Layered system architecture of PersonaAI	17
4.4	Sequence diagram: answering a question from the notes	18
4.5	Activity diagram: quiz generation, grading and topic-memory update	20
4.6	Use case diagram	22
4.7	Data flow diagram, level 0 (context)	23
4.8	Data flow diagram, level 1	23
4.9	Entity–relationship diagram of the SQLite database	24
6.1	Development set: grading of answers with and without the notes	38
6.2	Held-out (test) set: grading of answers with and without the notes	38
6.3	Simulated mastery of the genuinely weak topics per quiz	41
6.4	Sign-in page	44
6.5	Home dashboard: welcome banner, statistics, today's plan and topics to revise (sample quiz history)	44
6.6	Notes Q&A: a real answer from the notes with file and page citations	45
6.7	Notes Q&A: an off-syllabus question is refused instead of answered from general knowledge	45
6.8	Quiz setup: documents, topics, number of questions and mode	46
6.9	Taking a quiz: one question at a time with progress and question navigator (real generated question)	46
6.10	Quiz results with per-question review, explanations and sources	47
6.11	Progress dashboard: score trend, weak and strong topics, accuracy per topic (sample quiz history)	48
6.12	Creating a study plan	49
6.13	Study plan timeline with task chips and weak-topic priority flags (sample quiz history)	49
6.14	Responsive layout on a phone-sized screen	50

LIST OF TABLES

Table No.	Title	Page No.
2.1	Summary of the literature survey	8
3.1	Comparison of existing study tools with PersonaAI	10
3.2	Hardware requirements and development machine	11
3.3	Software used, with versions	12
3.4	Functional requirements	12
3.5	Non-functional requirements	13
4.1	Dataset summary	15
4.2	Sample records from the evaluation question set (reference answers shortened)	15
4.3	Model choices and justification	18
4.4	Database tables	24
4.5	Baselines used in the evaluation	25
4.6	Components used versus recent alternatives	25
5.1	Inference and configuration setup	26
5.2	How the design parameters were chosen	27
5.3	Development phases	34
5.4	Automated test suite (101 tests, all passing)	35
5.5	Sample test cases	35
5.6	Defects found during testing and their fixes	36
6.1	Answer accuracy with and without the notes (held-out rows are the test result)	38
6.2	Cross-tabulation of RAG verdict against no-RAG verdict (number of questions)	38
6.3	Accuracy by unit	39
6.4	Answer length and citations	39
6.5	Observed response times on the development machine	39
6.6	Simulation results: adaptive versus even-mix quizzes (200 simulated students per mode)	40
6.7	Analysis of notes-grounded answers that were not fully correct	42
6.8	Ablation study	43

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the (ROC) Curve
BKT	Bayesian Knowledge Tracing
BM25	Best Matching 25 (keyword ranking function)
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DKT	Deep Knowledge Tracing
DOCX	Office Open XML word-processing document
ER	Entity–Relationship
GPU	Graphics Processing Unit
HDFS	Hadoop Distributed File System
HLR	Half-Life Regression
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MCQ	Multiple-Choice Question
MRR	Mean Reciprocal Rank
NLP	Natural Language Processing
OCR	Optical Character Recognition
PDF	Portable Document Format
Q&A	Question and Answer
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
SDG	Sustainable Development Goal
SQL	Structured Query Language
TXT	Plain-text file
UI	User Interface
UML	Unified Modeling Language

CHAPTER 1
INTRODUCTION

1.1 Background

Large language models (LLMs) have made conversational AI assistants freely available to students. They explain concepts fluently and can be asked follow-up questions at any hour, which makes them attractive as study aids [1]. However, an LLM answers from its general training data, which may not match how a particular course defines or teaches a topic, and it occasionally produces plausible but incorrect statements, a behaviour known as hallucination [2]. A general assistant also has no memory of an individual learner's performance: a student who repeatedly struggles with one topic receives the same generic help as everyone else.

Retrieval-Augmented Generation (RAG) addresses the first two problems by retrieving relevant passages from a document collection and asking the model to answer from them [3]. Learning research addresses the third: practice testing and spaced review are among the most effective study techniques known [4], [5], [6], and they work best when practice is directed at what the learner does not yet know.

PersonaAI combines these ideas in one application. It grounds every answer in the student's uploaded notes and cites the file and page, turns the same notes into quizzes, remembers which topics each student answers incorrectly, and uses that memory to focus later quizzes and to build a personalised revision plan for an upcoming examination. The project was developed and evaluated with the material of a Big Data course (Units III and IV: Apache Pig, Flume, Hive and Sqoop).

1.2 Motivation

- Examination revision usually happens from several sources (lecture notes, unit PDFs, question banks), and finding the right page for a specific question takes time.
- Answers that do not match the syllabus can mislead students: an examination expects the points that were taught, not a generic textbook answer.
- Practice testing and spaced review are effective [6], but organising them by hand across many topics is tedious, so students rarely do it.

- Students often do not know which topics they are weak in until they lose marks in an examination.
- Many students cannot rely on paid AI services or a continuous internet connection; a tool that also works offline with a local model is more inclusive. This connects the project to United Nations Sustainable Development Goal 4, Quality Education [7].

1.3 Problem Statement

To design, build and evaluate a web application that (i) answers a student's questions using only that student's uploaded course material, with verifiable file-and-page citations, and says so when the material does not contain the answer; (ii) generates multiple-choice quizzes from the same material and grades them; (iii) maintains a per-student, per-topic model of strengths and weaknesses from quiz results; and (iv) uses that model to target later quizzes and to produce a day-by-day study plan for an examination date, while remaining usable when the online AI service is rate-limited or unavailable.

1.4 Objectives

- Ingest PDF, DOCX and TXT files, extract text page by page, and build a searchable vector index for each student.
- Answer questions from the notes only, citing the source file and page, and reply "not found in your notes" for questions the notes do not cover.
- Generate valid multiple-choice quizzes (5, 10 or 15 questions) with explanations, and save every answer with its topic.
- Track each topic's accuracy over recent answers and classify topics as weak, learning or strong.
- Provide an adaptive quiz mode that concentrates practice on weak topics.
- Generate a study plan that covers every topic, gives weak topics more time, and changes as performance improves.
- Evaluate answer accuracy with and without retrieval on real examination questions using a held-out set, and evaluate the adaptive quiz strategy against an even spread of questions.

- Provide a clean, responsive user interface that works in desktop and mobile browsers.

1.5 Scope and Limitations

In scope: study material for one or more subjects per student; private per-user memory; the four features (Notes Q&A, quizzes, weak-topic memory with a progress dashboard, study plan); an offline mode using a local model; and a quantitative evaluation of the approach.

Out of scope: lecture recording and transcription, calendar and e-mail integration, a native mobile app, payments, password-based authentication and multi-institution deployment.

Known limitations (discussed in Chapters 6 and 7): no model was trained or fine-tuned; all models are pretrained and used for inference only. Scanned, image-only PDFs are not supported because no OCR is performed. The evaluation uses one course and 60 questions, and the benefit of adaptive quizzes was measured by simulation rather than with real students. Sign-in is by username only, which is suitable for a classroom demonstration but not for public deployment.

1.6 Organization of the Report

Chapter 2 reviews the literature on retrieval-augmented generation, retrieval and re-ranking, evaluation with LLM judges, question generation and learner modelling, and identifies the research gap. Chapter 3 analyses existing approaches, feasibility and the hardware and software used. Chapter 4 describes the data, the overall pipeline, the system architecture and the methodology, together with the baselines and recent alternative models. Chapter 5 covers the implementation: configuration, parameter choices, key code, the user interface and testing. Chapter 6 presents the evaluation results, error analysis, ablation experiments, screenshots and an ethics discussion. Chapter 7 concludes the report and outlines future work.

CHAPTER 2
LITERATURE SURVEY

This chapter reviews work in five areas that PersonaAI builds on: retrieval-augmented generation; dense retrieval and re-ranking; the behaviour and evaluation of LLM-based systems; automatic question generation and LLMs in education; and learner modelling, spaced repetition and long-term memory. A summary is given in Table 2.1, and the research gap is stated in Section 2.3.

2.1 Review of Relevant Work

2.1.1 Retrieval-Augmented Generation

Lewis et al. [3] introduced Retrieval-Augmented Generation, in which a neural retriever selects passages from a large document collection and a sequence-to-sequence generator conditions its answer on them. Trained end to end on Wikipedia, RAG set the state of the art on three open-domain question-answering tasks at the time and produced more specific and factual text than a model relying only on its parameters. RAG also makes answers traceable to sources and can be updated by changing the collection rather than retraining. PersonaAI applies the same idea with the student's notes as the collection, but uses an instruction-following LLM without fine-tuning, and adds an explicit instruction to refuse when the passages do not contain the answer.

Gao et al. [8] survey the rapidly growing RAG literature and organise it into *naive* RAG (retrieve, then generate), *advanced* RAG (improved indexing, query rewriting and re-ranking) and *modular* RAG (pipelines with interchangeable components). In this taxonomy PersonaAI is an advanced RAG system with a re-ranking stage. Asai et al. [9] proposed Self-RAG, which fine-tunes a model to decide when to retrieve and to critique its own output with special reflection tokens; their 7B and 13B models outperformed ChatGPT and retrieval-augmented Llama 2-chat on several question-answering and fact-verification tasks and improved citation accuracy. Self-RAG requires fine-tuning the generator, which is not possible with a hosted API model such as Gemini, so PersonaAI implements a simpler form of self-assessment: a score threshold on the re-ranker and a `NOT_FOUND` reply.

2.1.2 Dense Retrieval, Sentence Embeddings and Re-ranking

Karpukhin et al. [10] showed that a dual-encoder trained on question–passage pairs (Dense Passage Retrieval) outperforms the keyword-based BM25 ranking function [11] by 9–19 percentage points in top-20 passage retrieval accuracy on open-domain benchmarks. Sentence-BERT [12] showed that Siamese transformer networks produce sentence embeddings whose cosine similarity reflects meaning, enabling fast semantic search; MiniLM [13] distils large transformers into small, fast models. The `all-MiniLM-L6-v2` model used in PersonaAI combines these results: it produces 384-dimensional embeddings and runs on a laptop CPU. One limitation, observed during this project, is that it reads at most 256 word-pieces, so longer passages are silently truncated.

Embedding search encodes the question and each passage independently. A cross-encoder instead reads the question and passage together and is far more accurate at judging relevance, at a higher cost per pair. Nogueira and Cho [14] re-ranked BM25 candidates with a BERT cross-encoder trained on the MS MARCO passage-ranking dataset [15] and improved the previous state of the art by 27% (relative) in MRR@10. PersonaAI uses this two-stage design: embedding search returns 20 candidates and a MiniLM cross-encoder trained on MS MARCO selects the best five. Recent general-purpose embedding models such as E5 [16] and BGE [17] are larger and stronger alternatives for the first stage (Section 4.8).

2.1.3 Behaviour and Evaluation of LLM-Based Systems

Ji et al. [2] survey hallucination in natural language generation, distinguishing *intrinsic* hallucinations (contradicting the source) from *extrinsic* ones (adding unverifiable content), and list retrieval grounding among the main mitigation strategies. Liu et al. [18] showed that LLMs use long contexts unevenly: accuracy on multi-document question answering is highest when the relevant passage is at the beginning or end of the context and drops when it is in the middle. This supports giving the model a small number of well-ranked passages (five in PersonaAI) rather than many.

Evaluating free-text answers is expensive. Zheng et al. [19] studied strong LLMs as judges on MT-Bench and Chatbot Arena and found that GPT-4's judgments agreed with human preferences over 80% of the time, similar to the agreement between

humans, while also documenting position, verbosity and self-enhancement biases. Es et al. [20] proposed RAGAs, a set of reference-free metrics (faithfulness, answer relevance and context relevance) computed with an LLM for evaluating RAG pipelines, validated against human judgments on their WikiEval dataset. PersonaAI's evaluation follows these lessons: the judge grades each answer against a written reference, the two candidate answers are shown in random order without labels, and columns are provided for manual overrides.

2.1.4 Question Generation and LLMs in Education

Kurdi et al. [21] systematically reviewed automatic question generation for educational purposes, covering studies published between 2015 and 2019. They found that most systems generate factual questions from text, that evaluation practice is inconsistent across studies, and that controlling question difficulty remains an open problem. These systems pre-date instruction-tuned LLMs, which can now write complete multiple-choice questions with distractors and explanations, but whose answer keys and distractors still need checking. Kasneci et al. [1] discuss the opportunities of LLMs in education (personalised explanations, feedback, practice material) together with risks such as incorrect output, bias and over-reliance, and argue for grounding and human oversight. PersonaAI's quiz module responds to both points: questions are generated only from the student's notes, validated by rules, and, for the weaker local model, checked by having the model answer its own questions without seeing the key.

2.1.5 Learner Modelling, Spaced Repetition and Long-Term Memory

Intelligent tutoring systems estimate what a learner knows in order to choose the next activity. Bayesian Knowledge Tracing [22] models the probability that a student has mastered each skill as a hidden state updated after every answer. Deep Knowledge Tracing [23] replaced this with a recurrent neural network and reported an AUC of 0.86 against 0.67 for BKT on the ASSISTments dataset, at the cost of needing large interaction logs and being harder to interpret. Settles and Meeder [24] introduced half-life regression, which estimates how long a word stays in a learner's memory; on Duolingo practice logs it reduced the error in predicting recall by more than 45% compared with baseline methods.

Roediger and Karpicke [4] showed that taking tests improves long-term retention more than restudying the same material (the testing effect), and Cepeda et al. [5] found across hundreds of experiments that spacing practice over time improves recall compared with massed practice. Dunlosky et al. [6] rated practice testing and distributed practice as the two most useful of ten common study techniques. Finally, recent work on long-term memory for chat assistants, such as the LongMemEval benchmark [25], shows that commercial assistants and long-context LLMs lose about 30% accuracy when information must be recalled across sustained interactions. PersonaAI does not try to remember conversations; its long-term memory is a structured, per-topic record of quiz performance stored in a database, which is simple, explainable and cannot hallucinate.

2.2 Summary of the Literature

Table 2.1 summarises the reviewed papers.

Table 2.1 Summary of the literature survey

Author / Year	Journal / Venue	Method	Dataset	Results	Limitations
Lewis et al., 2020 [3]	NeurIPS	Neural retriever + seq2seq generator trained end to end	NQ, TriviaQA, WebQuestions, CuratedTrec, MS MARCO, FEVER	State of the art on three open-domain QA tasks at the time; more factual generation	Wikipedia-scale index; joint training; no explicit abstention
Karpukhin et al., 2020 [10]	EMNLP	Dual-encoder dense passage retrieval	NQ, TriviaQA, WebQ, CuratedTREC, SQuAD	+9–19 points top-20 retrieval accuracy over BM25	Needs labelled question–passage pairs
Nogueira & Cho, 2019 [14]	arXiv	BERT cross-encoder re-ranking of BM25 candidates	MS MARCO, TREC-CAR	27% relative gain in MRR@10 on MS MARCO	Cost grows with candidates; only re-ranks what stage one finds
Gao et al., 2023 [8]	arXiv (survey)	Survey of naive, advanced and modular RAG	Literature	Taxonomy of retrieval, augmentation, generation and evaluation methods	No new experiments; fast-moving field
Asai et al., 2024 [9]	ICLR	Self-RAG: model learns when to retrieve and critiques output	Open-domain QA, reasoning, fact verification, long-form	Beat ChatGPT and retrieval-augmented Llama 2-chat on several tasks	Requires fine-tuning the generator
Liu et al., 2024 [18]	TACL	Controlled study of position of relevant information	Multi-document QA, key-value retrieval	Accuracy highest at start/end of context, lower in middle	Analysis only; no remedy proposed
Es et al., 2024 [20]	EACL (Demos)	Reference-free LLM metrics for RAG (RAGAs)	WikiEval	Faithfulness and relevance metrics validated against human judgments	Depends on the scoring LLM
Zheng et al., 2023 [19]	NeurIPS (D&B)	Strong LLMs as judges	MT-Bench, Chatbot Arena	GPT-4 agreed with humans over 80%, similar to human–human agreement	Position, verbosity, self-enhancement bias
Ji et al., 2023 [2]	ACM Computing Surveys	Survey of hallucination in NLG	Literature	Taxonomy, metrics and mitigation (incl. grounding)	Largely pre-dates current LLMs
Kasneci et al., 2023 [1]	Learning and Individual Differences	Position paper on LLMs in education	—	Opportunities and risks; calls for oversight	No empirical evaluation
Kurdi et al., 2020 [21]	Int. J. AI in Education	Systematic review of question generation	Studies from 2015–2019	Mostly factual questions; inconsistent evaluation; difficulty control open	Pre-LLM systems
Piech et al., 2015 [23]	NeurIPS	Recurrent network for knowledge tracing (DKT)	ASSISTments, Khan Academy, simulated	AUC 0.86 vs 0.67 for BKT on ASSISTments	Needs large logs; hard to interpret
Settles & Meeder, 2016 [24]	ACL	Half-life regression for spaced repetition	Duolingo practice logs	Over 45% lower error in predicting recall	Word-level, language-learning domain
Wu et al., 2025 [25]	ICLR	Benchmark of long-term chat-assistant memory	500 questions in scalable chat histories	About 30% accuracy drop over sustained interactions	Conversational memory, not learner performance

2.3 Research Gap Identified

The reviewed work leaves the following gaps, which this project addresses at the scale of a mini project:

- **Combination of features.** RAG systems answer questions; question-generation systems write questions; learner models and spaced-repetition schedulers decide what to practise. Few student-facing tools connect all three, so that the questions a student gets wrong change what they are quizzed on next and how their revision time is planned.
- **Honest evaluation on a real course.** Published RAG results are mostly on Wikipedia-based benchmarks. There is little evidence on how a small, CPU-friendly RAG pipeline performs on a single course's own notes and examination questions, compared with the same LLM answering without notes, with abstentions counted rather than hidden.
- **Low-resource operation.** Methods such as Self-RAG or DKT need fine-tuning or large datasets. A study assistant for students needs components that run on a laptop or the free tier of an API, with a fully offline fallback.
- **Explainable learner memory.** Neural knowledge tracing is accurate but opaque; a student-facing tool benefits from a learner model whose decisions ("this topic is weak because you got 1 of the last 3 right") can be shown directly.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing Approaches and their Drawbacks

Without a dedicated tool, a student revises by reading PDFs, searching them for keywords, asking a general chatbot and occasionally attempting question-bank questions. The drawbacks are that answers are not tied to the syllabus, there is no record of which topics were answered wrongly, practice is not targeted, and revision across many topics before an examination is planned by hand. Table 3.1 compares the main categories of existing tools.

Table 3.1 Comparison of existing study tools with PersonaAI

Tool / approach	Answers from own notes	Citations	Quizzes	Tracks weak topics	Study plan
General AI chatbots	No (general knowledge)	No	On request	No	Generic
Notes-grounded assistants (e.g. NotebookLM)	Yes	Yes	Limited	No	No
Flashcard apps (e.g. Anki)	Cards written by the user	—	Flashcards	Per card	Review schedule
Quiz platforms (e.g. Quizlet)	From user's sets	—	Yes	Limited	No
PersonaAI (this project)	Yes	File + page	Auto-generated MCQs	Per topic	Day-by-day, weighted

The comparison reflects the general design of each category at the time of writing; individual products change frequently. The gap PersonaAI addresses is the combination of notes-grounded answers, automatic quizzes and a learner model that feeds back into both quizzes and planning, in one tool that also works offline.

3.2 Feasibility Study

Technical feasibility. All components are mature open-source libraries or documented APIs: Streamlit for the web interface [26], ChromaDB for vector search [27], sentence-transformers for local embeddings and re-ranking [28], SQLite for application data, the Google Gemini API for generation [29], and Ollama for local models [30]. The embedding and re-ranking models run on a laptop CPU.

Economic feasibility. The project uses only free software and the free tier of the Gemini API. The free tier is rate-limited (observed during development: 20 requests per day for `gemini-3.8-flash` and 15 requests per minute for `gemini-3.5-flash-lite`). The system handles this by waiting out short limits, falling back to the lighter model, and finally to a local Ollama model, so no paid service is required.

Operational feasibility. Students need only a web browser. Sign-in is by username, uploads are drag-and-drop, and every page works on phones as well as laptops.

Schedule feasibility. The work was planned in eight phases (setup, four features, evaluation, polish and documentation), each ending with a working, tested version (Section 5.5), which kept the scope achievable within the semester.

3.3 Hardware and Compute Used

No model was trained or fine-tuned in this project, so no GPU training time was needed. All compute was used for inference: embedding and re-ranking on the CPU, local LLM generation on the laptop GPU through Ollama, and online generation through the Gemini API. Table 3.2 lists the minimum requirements and the development machine; observed run times are reported in Table 6.5.

Table 3.2 Hardware requirements and development machine

Item	Minimum	Development machine used
Processor	Intel Core i5 or equivalent	AMD Ryzen 7 7445HS
RAM	8 GB (16 GB for offline mode)	16 GB
GPU	Not required (speeds up offline mode)	NVIDIA GeForce RTX 4050 Laptop GPU (used by Ollama)
Storage	About 6 GB (models + data)	SSD
Operating system	Windows 10/11, Linux or macOS	Windows 11 Home
Network	Needed for the Gemini API; not needed in offline mode	Broadband
Training time	Not applicable (pretrained models only)	Not applicable

3.4 Software Requirements

Table 3.3 lists the software used, with the versions installed in the project's virtual environment.

Table 3.3 Software used, with versions

Component	Technology	Version
Language	Python	3.12.10
Web UI	Streamlit (custom theme, st.navigation)	1.65.0
LLM (online)	Google Gemini API via google-genai (gemini-3.8-flash, gemini-3.5-flash-lite)	2.28.0
LLM (offline)	Ollama server with llama3.1:8b; ollama Python client	0.40.10; 0.6.3
Embeddings and re-ranking	sentence-transformers: all-MiniLM-L6-v2, cross-encoder/ms-marco-MiniLM-L-6-v2	6.1.0
Deep learning runtime	PyTorch; Hugging Face transformers	2.14.1; 5.19.0
Vector database	ChromaDB (persistent, cosine distance)	1.5.9
Application database	SQLite (Python standard library)	—
Document parsing	pypdf; python-docx	6.19.0; 1.2.0
Charts and data	Plotly; pandas	7.1.0; 3.0.6
Testing	pytest; Streamlit AppTest	9.1.1
Development environment	Visual Studio Code, Python virtual environment (venv), Windows 11	—
Version control	Git	2.52.0

3.5 Functional and Non-Functional Requirements

The functional requirements are listed in Table 3.4 and the non-functional requirements in Table 3.5.

Table 3.4 Functional requirements

ID	Requirement
FR1	The user signs in with a username; each username has a private space (documents, quizzes, plan).
FR2	The user uploads one or more PDF, DOCX or TXT files with an optional subject.
FR3	The system extracts text page by page, chunks it, embeds it and stores it in a vector index tagged with user, file and page.
FR4	The system extracts 5–15 syllabus topics per document; the user can edit them.
FR5	The user asks questions in a chat; answers come only from the notes and cite file and page; questions the notes do not cover get “not found”.
FR6	The user generates a quiz of 5, 10 or 15 MCQs from chosen documents and topics, in adaptive or even-mix mode.
FR7	The system validates generated questions and regenerates missing ones; options are shuffled.
FR8	The user answers one question at a time, submits, and sees the score, correct answers, explanations and sources.
FR9	Every answer is saved with its topic; topic scores and weak/strong status are updated after each quiz.
FR10	A progress page shows quiz history, per-topic accuracy and weak and strong topics.
FR11	The user creates a study plan from an exam date, hours per day and documents; marks days done; regenerates it with the latest scores.
FR12	If the online model fails, the system falls back to a lighter online model and then to a local model.

Table 3.5 Non-functional requirements

Category	Requirement
Accuracy and honesty	Answers must be traceable to the notes; the assistant must say when the notes do not contain an answer.
Performance	A Q&A answer within a few seconds on the online model; a 10-question quiz within about 20 s.
Availability	Usable when the online API is rate-limited or offline (local model fallback).
Usability	Clean, consistent UI; works at desktop and phone widths; clear empty states and error messages.
Privacy	Each user's documents and results are filtered by user id; API keys are kept in a git-ignored .env file.
Maintainability	Modular code (UI separate from logic), automated tests, one module per responsibility.
Portability	Runs with `streamlit run app.py` on Windows, Linux or macOS with Python 3.12.

CHAPTER 4
DATASET, DESIGN AND METHODOLOGY

4.1 Overall Pipeline

Fig. 4.1 shows the complete pipeline. Lane A runs once per uploaded file: text is extracted page by page, cleaned, split into passages (chunks), embedded and stored in ChromaDB. Lane B runs for every question or quiz: the most relevant chunks are retrieved and re-ranked, and an LLM produces a cited answer or a set of validated quiz questions, whose results update the student's topic scores. Lane C is the evaluation: questions from the course question bank, which is deliberately kept out of the index, are answered with and without the notes and graded by a blind LLM judge. The application is deployed as a Streamlit web app.

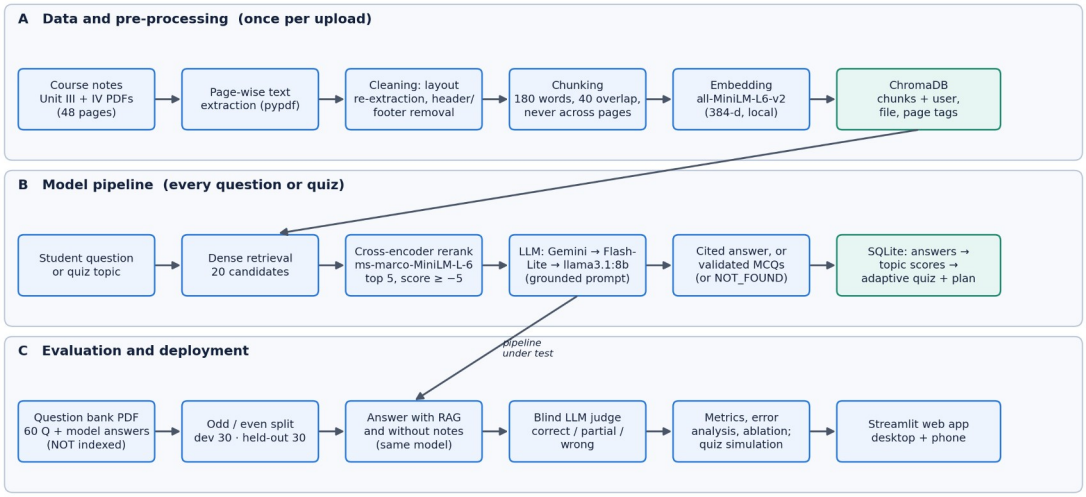


Fig. 4.1 Overall pipeline of PersonaAI: data, pre-processing, models, evaluation and deployment

4.2 Dataset

PersonaAI does not train a model, so its "dataset" has two parts: the **course material** that the system retrieves from, and the **evaluation question set** used to measure answer quality. Both come from the Big Data course (Units III and IV). The adaptive-quiz evaluation uses **simulated students** instead of real quiz logs (Section 6.4). Table 4.1 summarises the data and Table 4.2 shows sample records.

Table 4.1 Dataset summary

Attribute	Details
Dataset name and source	Big Data course material, Units III and IV (Apache Pig, Flume, Hive, Sqoop), provided by the course instructor: BIG_DATA_UNIT-III.pdf, BIG_DATA_UNIT-4.pdf (notes) and BD MID 2 SHORTS.pdf (question bank)
Licence	Internal course material used for academic evaluation only; not redistributed with the code [confirm permission with the guide]
Notes indexed for retrieval	2 PDFs, 48 pages (27 + 21), 7,276 words, 48 chunks; 26 syllabus topics (13 per unit) extracted by the LLM
Evaluation questions	60 short examination questions with model answers from the question bank (19 pages): 30 Unit III, 30 Unit IV
Target / classes	Each answer is graded into one of three classes: correct, partial or wrong ("not found" counts as wrong)
Class distribution	Balanced by unit: each split has 15 Unit III and 15 Unit IV questions
Development / held-out split	30 / 30 (odd- / even-numbered questions); no training split, because nothing is trained
Pre-processing	Page-wise extraction, layout re-extraction of one-word-per-line pages, removal of repeated headers/footers, 180-word chunks with 40-word overlap
Simulated students (quiz study)	200 per mode; 20 topics, 5 genuinely weak (correct-answer probability 25–45%, others 70–95%); 10 quizzes of 10 questions; seeds 0–199

Table 4.2 Sample records from the evaluation question set (reference answers shortened)

ID	Unit	Question	Reference answer (excerpt)	Source
1	III	What is Apache Pig?	A high-level platform for processing large datasets, designed to work with Hadoop; uses Pig Latin instead of SQL.	BD MID 2 SHORTS.pdf Q1
2	III	What is Pig Latin?	A high-level scripting language for Hadoop following a data-flow model; the output of one operation is the input to the next.	BD MID 2 SHORTS.pdf Q3
56	IV	What is Bucketing in Hive?	Divides table or partition data into a fixed number of files (buckets) using a hash of a column; improves joins and sampling.	Question bank

The reference answer for question 56 in Table 4.2 is paraphrased from the question bank; all 60 questions and reference answers are stored in `evaluation/questions.csv` and `evaluation/questions_holdout.csv`.

4.3 Pre-processing

Fig. 4.2 shows the ingestion pipeline. Text is extracted page by page with pypdf; DOCX and TXT files, which have no pages, are split into sections of about 400 words. Two problems found in the real course PDFs were handled explicitly:

- **One word per line.** One PDF extracted with every word on its own line, which breaks sentence-level meaning. When the average line on a page has fewer than

1.5 words, the page is re-extracted in pypdf's layout mode, which restores normal lines.

- **Repeated headers and footers.** Every page of two PDFs began with a "Prepared by … Page N" line, which polluted search results. Lines among the first or last three lines of a page that repeat on at least half of the pages (ignoring digits) are removed. Restricting the rule to page edges was necessary: a first version also removed the labels "Answer:" and "Example:", which repeat inside the body of the question-bank PDF.

Pages are then split into chunks of 180 words with a 40-word overlap, never crossing a page boundary, so that every chunk has exactly one page to cite; a tail shorter than 30 words is merged into the previous chunk. The chunk size was chosen because the embedding model reads only 256 word-pieces (about 190 English words): the 500-token chunks originally planned would have been silently truncated. For the course notes, each page fits within one chunk, giving 48 chunks. Finally, the LLM lists 5–15 syllabus topics for each document (from at most 24,000 characters of its text); the student can edit this list, and quiz answers are recorded against these topics.

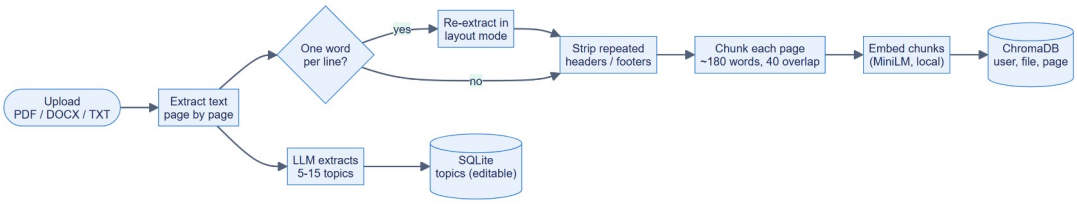


Fig. 4.2 Document ingestion and pre-processing pipeline

4.4 Development / Held-out Split and Avoiding Data Leakage

Because no model is trained, there is no training set; the risk of leakage is instead that the system could see the answers, or that design choices could be tuned to the questions used for the final result. The following measures were taken:

- **The answer source is not indexed.** The evaluation questions and reference answers come from the question-bank PDF, and only the two unit PDFs were indexed for retrieval, so RAG could not copy the reference answers.
- **Development and held-out halves.** The 60 questions were split into odd-numbered (development) and even-numbered (held-out) halves, each balanced by

unit. The development set was used to find problems and to try changes such as keyword search; the held-out set was run exactly once, on the final pipeline.

- **Parameters fixed before evaluation.** The chunk size, re-ranker threshold and other retrieval parameters were set during feature development (Phase 1), before the evaluation set was built (Phase 5).
- **Fixed model.** Every answer and every judgment in a run used the same model (`gemini-3.5-flash-lite`), with automatic fallback disabled, so results are not mixed across models.
- **Blind grading.** The judge saw the RAG and no-RAG answers as "Answer 1" and "Answer 2" in random order. The remaining risk, that the judge is the same model that wrote the answers, is discussed in Section 6.5.

4.5 System Architecture and Model Choice

PersonaAI follows a layered architecture (Fig. 4.3). The presentation layer is a Streamlit application: `app.py` shows the sign-in card and routes between five pages in `views/`, using shared styling in `ui/`. The application layer (`core/`) contains all logic and no user-interface code, so it can be tested directly. The data layer stores application data in SQLite and chunk embeddings in ChromaDB. The AI-model layer has two small local models (embedding and re-ranking) and the generative models, which are reached only through `core/llm.py`.

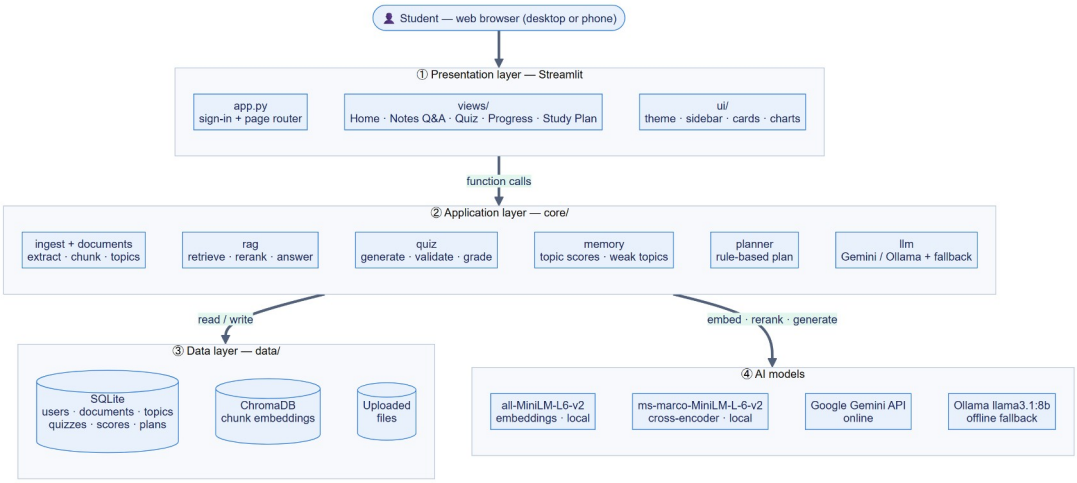


Fig. 4.3 Layered system architecture of PersonaAI

Fig. 4.4 shows how these components interact when a student asks a question, including the "not found" branch and the model fallback chain. The choice of each model is justified in Table 4.3.

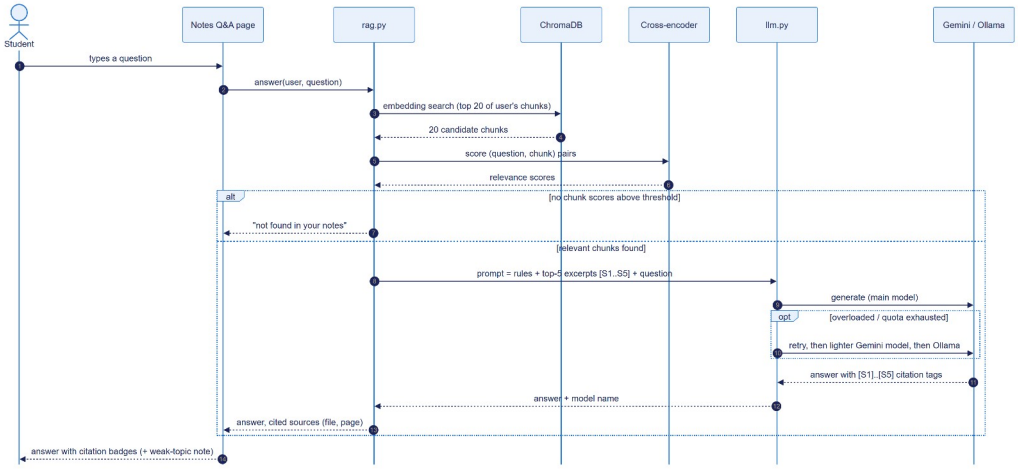


Fig. 4.4 Sequence diagram: answering a question from the notes

Table 4.3 Model choices and justification

Component	Model used	Justification
Embedding (stage 1)	all-MiniLM-L6-v2 (sentence-transformers, 384-d)	Small and fast on a CPU; strong general-purpose semantic search [12], [13]; runs locally, so notes are not sent anywhere for indexing.
Re-ranker (stage 2)	cross-encoder/ms-marco-MiniLM-L-6-v2	Reads question and passage together, which fixed cases where embedding search ranked the defining passage low [14]; small enough for a laptop.
Generator (online)	Gemini: gemini-3.8-flash, then gemini-3.5-flash-lite [31]	Strong instruction following and JSON output on a free tier; the lighter model has higher rate limits.
Generator (offline)	Llama 3.1 8B via Ollama [32]	Open-weight model that runs on the development laptop's GPU; keeps the app usable without internet or quota.
Learner model	Accuracy over the last 10 answers per topic, with thresholds	Explainable to students, needs no training data, and adapts quickly when a student improves (Section 4.6.3).
Planner	Rule-based weighted allocation	Guarantees full topic coverage and spacing, uses no API quota, and is testable.

4.6 Methodology

4.6.1 Grounded Question Answering with Abstention

For a question q, embedding search returns the 20 chunks with the highest cosine similarity between the question embedding $\mathbf{e}(q)$ and chunk embedding $\mathbf{e}(c)$, restricted to the student's own documents:

$$sim(q, c) = e(q) \cdot e(c) / (\|e(q)\| \|e(c)\|) \quad (4.1)$$

The cross-encoder then gives each candidate a relevance score $s(q, c)$, and the answer is generated from the set S of at most five candidates with the highest scores that also pass the threshold $\tau = -5$:

$$S = \text{top-5}\ \{\, c \in C_{20} : s(q, c) \geq \tau \,\}, \quad \tau = -5 \tag{4.2}$$

The threshold was calibrated on the course notes: questions on the syllabus scored between −2.3 and 8.6 for their best chunk, while unrelated questions scored between −9.4 and −4.7. If S is empty, the system replies "not found in your notes" without calling the LLM, saving quota. Otherwise the selected excerpts are numbered [S1]… [S5] with their file and page, and the LLM is instructed to answer only from them, cite them inline, write in the style of an examination answer, and reply with the token `NOT_FOUND` if they do not contain the answer. Near-domain questions can pass the threshold, so this second, model-level check is needed. Citations are parsed from the reply and shown as file-and-page badges with the excerpts available underneath.

4.6.2 Quiz Generation and Validation

A quiz is described by an *allocation* {topic → number of questions}. The best excerpts for each topic are retrieved (at most 14 in total, to fit a local model's context), and one LLM call asks for exactly the allocated number of questions per topic as JSON: question, four options, answer letter, explanation, topic tag and source tag. Every question is validated by rules (Section 5.3.5); missing questions are requested again, up to three calls per quiz, and options are shuffled so that the correct answer is not always where the model tends to put it. For the local model, an additional answer-key check asks the model to answer its own questions without seeing the key and discards questions where its answer disagrees. Fig. 4.5 shows the full flow from setup to the update of topic scores; adaptive and even-mix quizzes differ only in the allocation step.

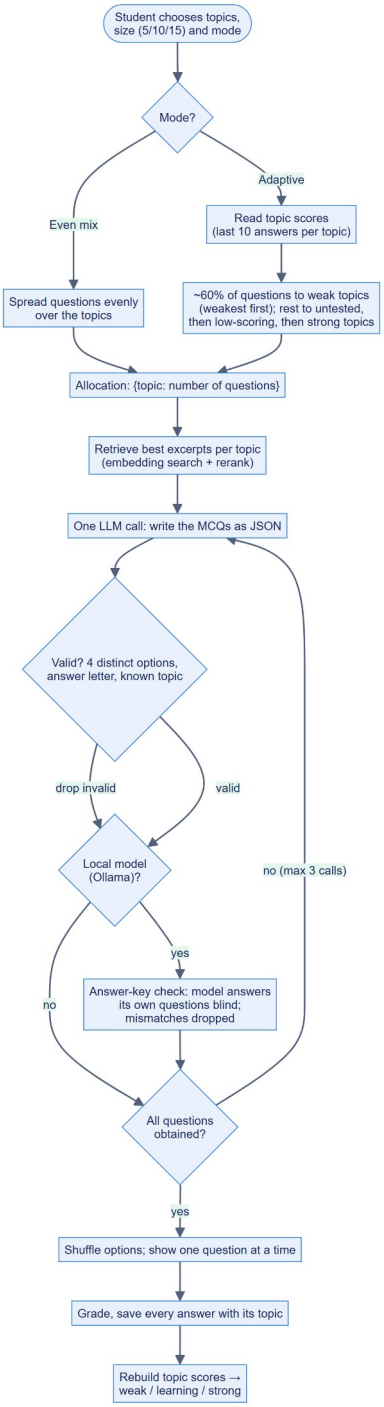


Fig. 4.5 Activity diagram: quiz generation, grading and topic-memory update

4.6.3 Weak-Topic Memory

Let R_t be the most recent answers of the student on topic t, up to ten. The topic's accuracy is

$$acc(t) = (1 / |R_t|) \cdot \Sigma_{a \in R_t} correct(a), \quad |R_t| \leq 10 \tag{4.3}$$

and its status is *untested* if the student has fewer than three answers on it, *weak* if acc(t) < 0.60, *strong* if acc(t) ≥ 0.80, and *learning* otherwise. Using recent rather than all-time accuracy lets a status change when the student improves: a student who answered a topic wrongly ten times and then correctly nine times would be at 47% all-time (weak) but at 90% over the last ten answers (strong). Topic scores are not updated incrementally; after every quiz they are rebuilt from the saved answers in one SQL statement, so they can never drift from the underlying data.

4.6.4 Adaptive Quiz Allocation

For an adaptive quiz of n questions, round(0.6 n) questions go to weak topics, weakest first, cycling if there are fewer weak topics than questions. The remaining questions go first to untested topics (to discover new weak spots), then to the lowest-scoring learning topics, and finally to strong topics seen longest ago, which gives a light spaced-review effect. If no topic is weak yet, all questions follow the second rule. The even-mix mode spreads the questions as evenly as possible over randomly ordered topics.

4.6.5 Study Planner

The planner is rule-based rather than LLM-generated, so that its guarantees always hold and it uses no API quota. Study time (study days × hours per day) is divided into 30-minute slots. Every topic first receives one slot; the remaining M slots are divided in proportion to topic weights w (weak 3, untested or learning 2, strong 1) using the largest-remainder method:

$$slots(t) = 1 + \lfloor M \cdot w(t) / \Sigma_u w(u) \rfloor \text{ (+1 for the largest remainders)} \qquad (4.4)$$

Each topic's time is split into a study session (about 60%) and a review session with a five-question quiz (about 40%) at least two days later, applying the spacing effect [5]. Days are filled greedily: due reviews first, then new topics in syllabus order. For examinations more than six days away, the last two days are a 15-question mock examination and a final revision of the weakest topics. Regenerating the plan re-runs these rules from the current day with the latest topic scores and lists how each topic's time changed.

4.6.6 System Models

The primary actor is the student; the LLM service is a secondary actor used for topic extraction, answering and quiz generation (Fig. 4.6). Fig. 4.7 shows the context-level data flow diagram and Fig. 4.8 decomposes the system into five processes and four data stores. Application data is stored in seven SQLite tables (Fig. 4.9, Table 4.4); foreign keys cascade on delete, so removing a document removes its topics. Chunk text and embeddings live in ChromaDB, with metadata (user id, document id, file name, page) used for filtering and citations.

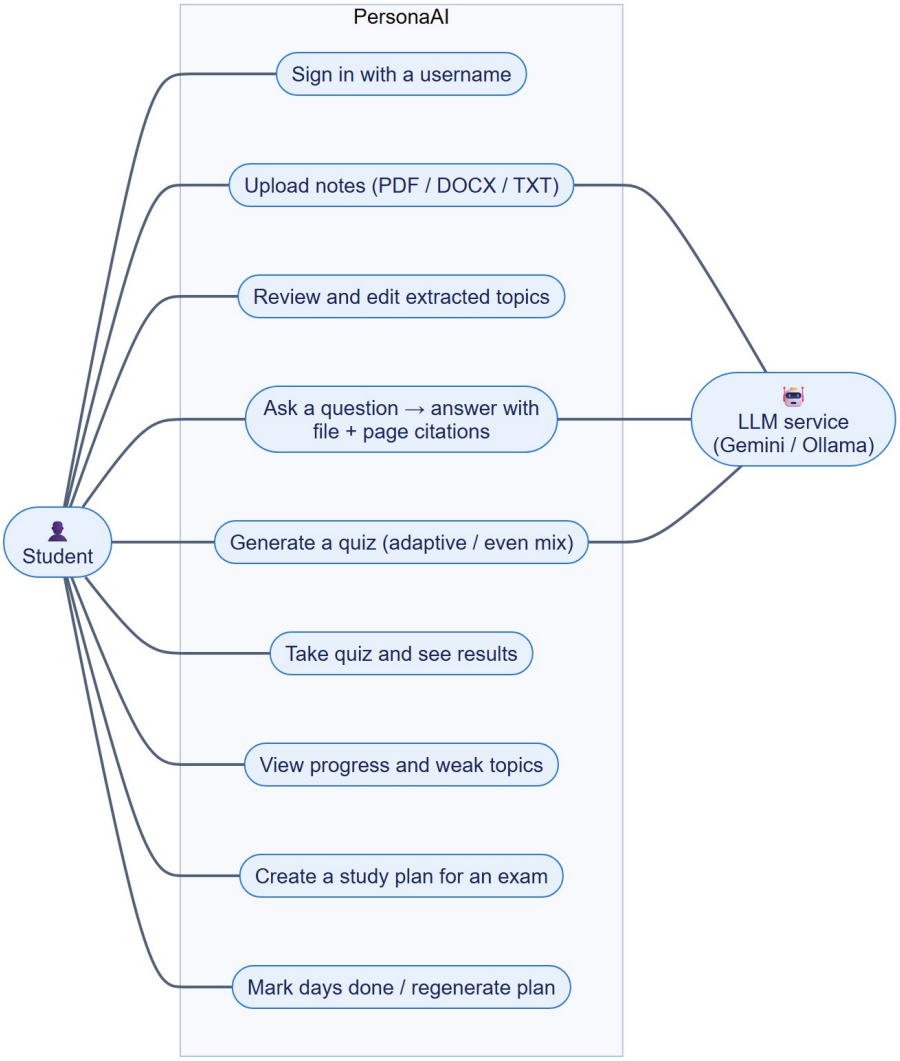


Fig. 4.6 Use case diagram

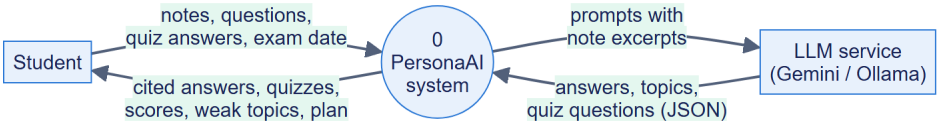


Fig. 4.7 Data flow diagram, level 0 (context)

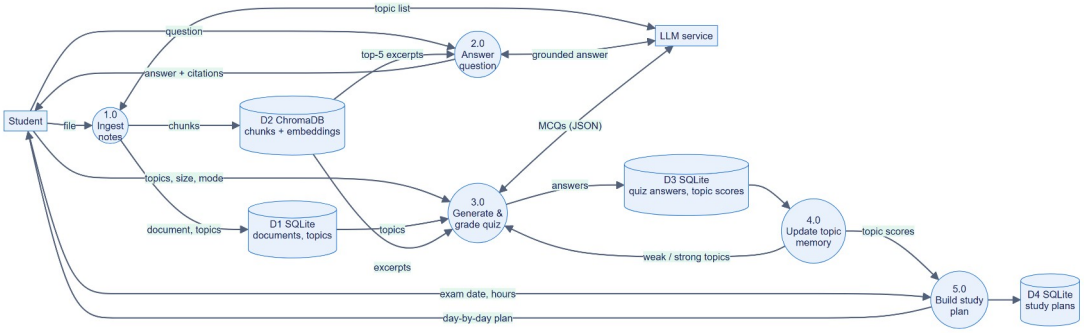


Fig. 4.8 Data flow diagram, level 1

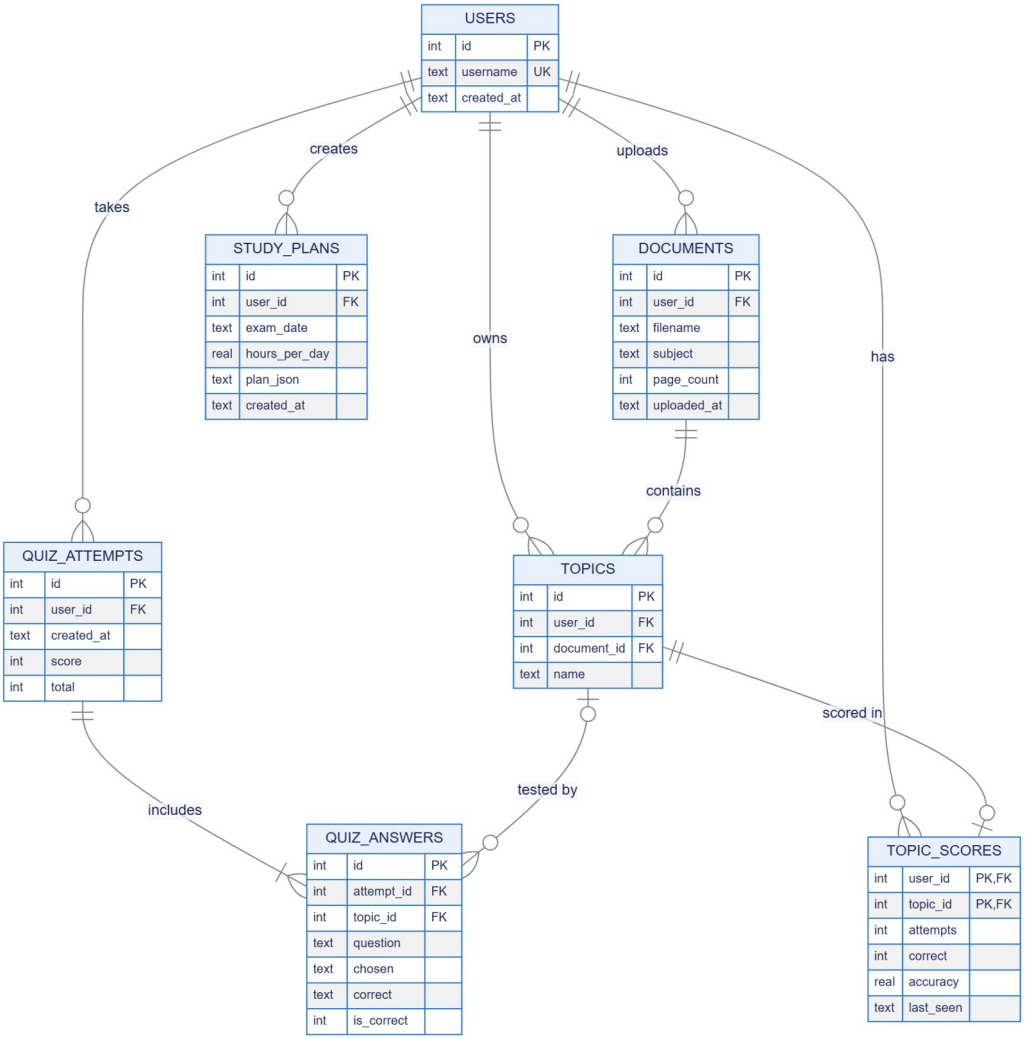


Fig. 4.9 Entity–relationship diagram of the SQLite database

Table 4.4 Database tables

Table	Purpose	Key columns
users	One row per username	id, username (unique, case-insensitive)
documents	Uploaded files	id, user_id, filename, subject, page_count
topics	Syllabus topics per document (editable)	id, user_id, document_id, name
quiz_attempts	One row per submitted quiz	id, user_id, created_at, score, total
quiz_answers	Every answer with its topic	attempt_id, topic_id, question, chosen, correct, is_correct
topic_scores	Derived accuracy per topic	user_id, topic_id, attempts, correct, accuracy, last_seen
study_plans	Saved plans (JSON) and completed days	id, user_id, exam_date, hours_per_day, plan_json

4.7 Baseline Models for Comparison

Each design decision that the evaluation tests is compared against a simpler baseline (Table 4.5).

Table 4.5 Baselines used in the evaluation

Baseline	What it is	Compared with	Where
B1: LLM without notes	The same model (gemini-3.5-flash-lite) answers from general knowledge with the same exam-style instructions	Full RAG pipeline	Section 6.3
B2: Hybrid first stage	Embedding search plus BM25 keyword search [11], then the same re-ranker	Embedding-only first stage	Section 6.6
B3: Larger re-ranker	ms-marco-MiniLM-L-12-v2 instead of L-6 (retrieval-only check)	L-6 re-ranker	Section 6.6
B4: Even-mix quizzes	Questions spread evenly over all topics	Adaptive allocation	Section 6.4
B5: All-time accuracy	Topic accuracy over all answers	Last-10-answers window	Section 4.6.3 (worked example)

4.8 Comparison with Recent (State-of-the-Art) Models

Table 4.6 places each component next to recent, stronger alternatives and explains why the simpler choice was made. Only the re-ranker alternative was measured in this project; the others are listed for context and as future work.

Table 4.6 Components used versus recent alternatives

Component	Used in PersonaAI	Recent alternatives	Reason for the choice
Retrieval embeddings	all-MiniLM-L6-v2	E5 [16], BGE [17] embedding models	Several times larger; slower on a CPU. Retrieval misses were mostly caused by chunk context, not embedding quality (Section 6.5).
Re-ranker	MiniLM-L-6 cross-encoder	MiniLM-L-12 cross-encoder (tested), larger rerankers	L-12 found the answer for one more question in 30 (Section 6.6), at higher memory and latency.
RAG design	Two-stage retrieve-rerank-generate (advanced RAG [8])	Self-RAG [9], modular RAG	Self-RAG needs a fine-tuned generator; not possible with a hosted API model.
Generator	Gemini Flash / Flash-Lite; Llama 3.1 8B offline	Larger frontier and open-weight models	Free-tier access and a model that fits the laptop GPU.
Learner model	Recent-accuracy thresholds	BKT [22], DKT [23], HLR [24]	Need many answers per student to fit; less explainable.
RAG evaluation	Blind LLM judge against reference answers	RAGAs metrics [20]	Reference answers were available, so direct correctness grading was possible.

CHAPTER 5
IMPLEMENTATION

5.1 Inference and Configuration Setup

PersonaAI uses pretrained models only, so the usual training settings (optimizer, learning rate, loss function, epochs, batch size, early stopping) do not apply. Table 5.1 lists the settings that do determine the system's behaviour and results, in the format of the department's training-setup table.

Table 5.1 Inference and configuration setup

Parameter	Value
Models	all-MiniLM-L6-v2 (embedding), ms-marco-MiniLM-L-6-v2 (re-ranker), gemini-3.8-flash → gemini-3.5-flash-lite → llama3.1:8b (generation); all pretrained, none fine-tuned
Optimizer, learning rate, loss, epochs, batch size	Not applicable: no training or fine-tuning was performed
Retrieval	20 candidates → re-rank → top 5; re-ranker threshold −5; cosine distance in ChromaDB
Chunking	180 words, 40-word overlap, minimum 30 words, never across pages
Generation temperature	0.2 for Q&A and topic extraction; 0.7 for quiz generation; 0 for quiz answer-key check and the evaluation judge
Local model context	8,192 tokens (Ollama num_ctx); request timeout 60 s
Evaluation model	gemini-3.5-flash-lite for every answer and judgment, fallback disabled
Random seeds	Simulation: students seeded 0–199 per mode; quiz option shuffling in the app is intentionally unseeded
Hardware and run time	AMD Ryzen 7 7445HS, 16 GB RAM, RTX 4050 Laptop GPU; indexing the notes ≈ 26 s; see Table 6.5
Framework and version	Python 3.12.10, PyTorch 2.14.1, sentence-transformers 6.1.0, ChromaDB 1.5.9, Streamlit 1.65.0

5.2 Parameter Selection Approach

There are no learned hyper-parameters; instead, the pipeline has design parameters. Each was either derived from a constraint or chosen by testing on the course notes and the development questions, never on the held-out set (Table 5.2).

Table 5.2 How the design parameters were chosen

Parameter	Value	How it was chosen
Chunk size / overlap	180 / 40 words	Derived from the embedding model's 256-token input limit
Candidates / top-k	20 / 5	Re-ranker cost on a CPU; few, well-ranked passages suit the LLM [18]
Re-ranker threshold	−5	Calibrated on on-syllabus vs unrelated questions (Section 4.6.1)
First-stage retrieval	Embedding only	BM25 hybrid tried on the development set; no gain, reverted (Section 6.6)
Re-ranker size	MiniLM-L-6	L-12 tried in a retrieval-only check; one question in 30 better, not adopted
Recent-answer window	10 answers	Design choice: lets a status change after a few correct answers
Weak / strong thresholds	< 60% / ≥ 80%	Design choice matching common pass/distinction bands; minimum 3 answers
Adaptive weak share	60%	Design choice: majority on weak topics while still exploring others
Planner weights	3 / 2 / 1	Design choice; tests check that weak > learning > strong time
Topic-match floor	0.22 similarity	Calibrated: the closest topic was correct for all 9 test questions

5.3 Key Code Snippets

This section explains the most important parts of the code. The listings are taken directly from the project source; the full code is in the GitHub repository (Appendix A).

5.3.1 Page-Bounded Chunking

Listing 5.1 splits each page into overlapping word windows. Because chunks never span pages, each chunk has exactly one page to cite, and a short tail is folded into the last chunk instead of creating a tiny one.

Listing 5.1 Page-bounded chunking (core/documents.py)

```
def chunk_pages(
    pages: list[Page], size: int = CHUNK_WORDS, overlap: int =
     CHUNK_OVERLAP
) -> list[Chunk]:
    """Split each page into overlapping word windows. Chunks never
     span pages,
    so every chunk has exactly one page to cite."""
    if overlap >= size:
        raise ValueError("overlap must be smaller than size")
    chunks: list[Chunk] = []
    for page in pages:
        words = page.text.split()
        if not words:
            continue
        start = 0
        while True:
            end = min(start + size, len(words))
            # Fold a short tail into this chunk instead of making a
     tiny one.
            if len(words) - end < MIN_CHUNK_WORDS:
                end = len(words)
            chunks.append(Chunk(" ".join(words[start:end]),
     page.number, len(chunks)))
            if end == len(words):
                break
            start = end - overlap
    return chunks
```

5.3.2 Two-Stage Retrieval

Listing 5.2 implements Equations (4.1) and (4.2). The re-ranker was added after testing on the real notes: for “What is Apache Pig?”, embedding search ranked the defining chunk 8th, because almost every chunk was “about Pig”, and the answer came back as “not found”; the cross-encoder ranked it 1st (score 7.4, against 1.9 for the next chunk).

Listing 5.2 Two-stage retrieval (core/rag.py)

```
def retrieve(
    user_id: int, question: str, document_ids: list[int] | None =
     None, k: int = TOP_K
) -> list[vectorstore.Hit]:
    """Best k relevant chunks for the question, best first (empty if
     none are relevant)."""
    candidates = vectorstore.search(user_id, question, k=CANDIDATES,
     document_ids=document_ids)
    if not candidates:
        return []
    scores = _cross_encoder().predict([(question, h.text) for h in
     candidates])
    for hit, score in zip(candidates, scores):
        hit.score = float(score)
    ranked = sorted(candidates, key=lambda h: h.score, reverse=True)
    return [h for h in ranked if h.score >= MIN_SCORE][:k]
```

5.3.3 Grounding Instructions

Listing 5.3 is the system prompt given to the LLM for every question. Excerpt tags were changed from [1] to [S1] after the model confused a question number in the notes ("22. What is bucketing?") with an excerpt number.

Listing 5.3 Grounding instructions given to the model (core/rag.py)

```
SYSTEM_PROMPT = f"""You are PersonaAI, a study assistant for a
      university student.
Answer the question using ONLY the numbered excerpts from the
      student's own notes.

Rules:
- Use only facts stated in the excerpts. Do not add outside
      knowledge.
- Cite the excerpts you used inline with their tags, like [S1] or
      [S2][S4], right after the
  sentence they support. Only use these [S#] tags for citations,
      never the notes' own numbering.
- If the excerpts do not contain the answer, reply with exactly
      {NOT_FOUND} and nothing else.
- Write the way a good exam answer reads: a one-line definition or
      direct answer first,
  then key points as bullets. Include a short example or syntax if
      the excerpts give one.
- Keep it concise."""
```

5.3.4 LLM Access, Retries and Fallback

All generation goes through `core/llm.py`, which tries the configured Gemini model, then the lighter Gemini model, then the local Ollama model. The free tier returned three kinds of failure, each handled differently (Listing 5.4):

- **Overload (HTTP 503/504):** retried once after a short back-off, then the next model is tried.
- **Per-minute rate limit (HTTP 429 with a short retry delay, e.g. 6 s):** the system waits and retries the same model. An earlier version treated this as an outage and fell back to the much slower local model.
- **Daily quota (HTTP 429 with a retry delay of hours):** the model is skipped for that period, so later calls do not waste attempts on it.

Listing 5.4 Retry policy for transient errors and rate limits (core/llm.py)

```
def _with_retries(fn, *args, attempts: int = 2):
    """Retry with exponential backoff (1s, 2s, ...) for transient
     errors like overload.

    Rate limits (HTTP 429) depend on how long Google says to wait: a
     per-minute limit
    (retry in ~6s) is waited out on the same model, which is far
     faster than falling
    back to a local model; a daily quota (retry in hours) is raised
     so the caller can
    skip the model."""
    failures = rate_waits = 0
    while True:
        try:
            return fn(*args)
        except LLMError:
            raise
        except Exception as exc:
            cooldown = _quota_cooldown(exc)
            if cooldown is not None:
                if cooldown > MAX_RATE_LIMIT_WAIT_S or rate_waits >=
     MAX_RATE_LIMIT_WAITS:
                    raise
                rate_waits += 1
                log.info("Rate limited; waiting %.0fs and retrying",
     cooldown + 1)
                time.sleep(cooldown + 1)
                continue
            failures += 1
            if failures >= attempts:
                raise
            wait = 2 ** (failures - 1)
            log.warning("LLM call failed (%s); retrying in %ss",
     exc, wait)
            time.sleep(wait)
```

5.3.5 Quiz Validation and Answer-Key Check

Each generated question must have exactly four distinct, non-empty options, no “All/None of the above”, an answer given as a letter or as the exact option text, and a known topic tag; numeric answers are rejected because models mix 0- and 1-based indices. Shuffling the options removes the model’s position bias: over 4,000 shuffles, each position held the correct answer 24–26% of the time. In manual checks, Gemini produced 15 of 15 questions with correct answer keys, but `llama3.1:8b` produced wrong keys in 2 of 5, so local-model quizzes pass through the check in Listing 5.5.

Listing 5.5 Answer-key check for local-model quizzes (core/quiz.py)

```
def verify(questions: list[Question], hits) -> list[Question]:
    """Have the model answer its own questions without seeing the
     key; keep only the
    questions where its answer matches. Catches wrong keys and
     ambiguous questions."""
    listing = "\n\n".join(
        f"{i}. {q.question}\n" + "\n".join(f"{LETTERS[j]}. {o}" for
     j, o in enumerate(q.options))
        for i, q in enumerate(questions, 1)
    )
    reply = llm.generate(
        VERIFY_PROMPT.format(excerpts=_excerpts(hits),
     questions=listing),
        json_mode=True,
        temperature=0,
    )
    try:
        data = json.loads(re.sub(r"^```(?:json)?|```$", "",
     reply.strip(), flags=re.M).strip())
        answers = {int(a["n"]): str(a["answer"]).strip().upper()[:1]
     for a in data["answers"]}
    except (ValueError, KeyError, TypeError):
        log.warning("Quiz verification reply unreadable; keeping no
     questions: %r", reply[:200])
        return []
    kept = [q for i, q in enumerate(questions, 1) if answers.get(i)
     == LETTERS[q.answer]]
    log.info("Quiz verification kept %d of %d questions", len(kept),
     len(questions))
    return kept
```

5.3.6 Rebuilding Topic Scores

Listing 5.6 implements Equation (4.3) in SQL. It recomputes every topic's statistics for a student from the saved answers, with accuracy taken over the ten most recent answers.

Listing 5.6 Rebuilding topic scores from recent answers (core/db.py)

```
_REFRESH_SCORES = """
INSERT INTO topic_scores (user_id, topic_id, attempts, correct,
     accuracy, last_seen)
SELECT q.user_id, a.topic_id, COUNT(*), SUM(a.is_correct),
       (SELECT AVG(r.is_correct) FROM (
            SELECT a2.is_correct FROM quiz_answers a2
            JOIN quiz_attempts q2 ON q2.id = a2.attempt_id
            WHERE a2.topic_id = a.topic_id AND q2.user_id =
     q.user_id
            ORDER BY a2.id DESC LIMIT {recent}) r),
        MAX(q.created_at)
FROM quiz_answers a JOIN quiz_attempts q ON q.id = a.attempt_id
WHERE q.user_id = ? AND a.topic_id IS NOT NULL
GROUP BY a.topic_id
""".format(recent=RECENT_ANSWERS)
```

5.3.7 Adaptive Allocation

Listing 5.7 Adaptive allocation of questions to topics (core/memory.py)

```
def adaptive_allocation(stats: list[TopicStat], n: int) -> dict[int,
     int]:
    """Spread n questions with ~60% on weak topics (weakest first).
     The rest go to
    untested topics first (to discover new weak spots), then the
     lowest-scoring ones,
    then strong topics seen longest ago (a light spaced-repetition
     touch)."""
    weak = sorted((s for s in stats if s.status == "weak"),
     key=lambda s: s.accuracy)
    others = sorted(
        (s for s in stats if s.status != "weak"),
        key=lambda s: (
            {"new": 0, "learning": 1, "strong": 2}[s.status],
            s.accuracy if s.accuracy is not None else 0,
            s.last_seen or "",
        ),
    )
    if not weak and not others:
        return {}
    if weak and others:
        weak_n = round(n * ADAPTIVE_WEAK_SHARE)
    else:
        weak_n = n if weak else 0

    allocation: dict[int, int] = {}
    for i in range(weak_n):
        tid = weak[i % len(weak)].topic_id
        allocation[tid] = allocation.get(tid, 0) + 1
    for i in range(n - weak_n):
        tid = others[i % len(others)].topic_id
        allocation[tid] = allocation.get(tid, 0) + 1
    return allocation
```

Listing 5.7 implements the rules of Section 4.6.4. The Q&A page also tells the student when a question is about one of their weak topics. The topic is found by embedding similarity between the question and every topic name; a fixed similarity cut-off failed calibration (correct pairs scored as low as 0.27 while sibling topics scored up to 0.70), but the single closest topic was correct for all nine test questions, so the closest topic is used with only a low floor (0.22) to reject unrelated questions.

5.3.8 Weighted Study-Plan Allocation

Listing 5.8 implements Equation (4.4); if there is not enough time to give every topic one slot, it tells the student the minimum hours per day needed.

Listing 5.8 Weighted allocation of study slots (core/planner.py)

```
def _allocate_slots(
    topics: list[TopicStat], total: int, study_days: int,
     hours_per_day: float
) -> dict[int, int]:
    """Split `total` slots by weight (largest remainder), at least
     one per topic."""
    if total < len(topics):
        needed = math.ceil(len(topics) * SLOT_HOURS /
     max(study_days, 1) / SLOT_HOURS) * SLOT_HOURS
        raise PlanError(
            f"Not enough time to cover all {len(topics)} topics at
     {hours_per_day} h/day. "
            f"Plan at least {needed} h/day, or pick fewer
     documents."
        )
    slots = {t.topic_id: 1 for t in topics}
    remaining = total - len(topics)
    weights = {t.topic_id: WEIGHTS[t.status] for t in topics}
    weight_sum = sum(weights.values())
    shares = {tid: remaining * w / weight_sum for tid, w in
     weights.items()}
    for tid, share in shares.items():
        slots[tid] += int(share)
    leftover = total - sum(slots.values())
    # Ties go to the heavier (weaker) topic.
    by_remainder = sorted(shares, key=lambda tid: (-(shares[tid] %
     1), -weights[tid]))
    for tid in by_remainder[:leftover]:
        slots[tid] += 1
    return slots
```

5.4 Deployment and User Interface

The application runs with `streamlit run app.py`. Settings are read from a git-ignored `.env` file locally, or from Streamlit secrets when hosted on Streamlit Community Cloud, so the API key is never stored in the code. `app.py` uses `st.navigation` with the page list hidden, and `ui/shell.py` draws a custom sidebar with the logo, icon navigation and a profile card that also shows which AI model is active.

The interface uses a light theme with white rounded cards, the Inter typeface for text and Poppins for headings. One primary colour (blue) is used for actions and charts, and green, amber and red are reserved for status (correct/partial/wrong, strong/learning/weak); status is always shown with an icon or label as well as colour. All styling lives in one module (`ui/styles.py`), user-provided text such as file and topic names is HTML-escaped before being inserted into custom components, and layouts reflow to a single column on phones. The quiz shows one question at a time;

answers are recorded through widget callbacks because Streamlit discards the state of widgets that are not displayed. Screenshots of every page are given in Section 6.7.

5.5 Development Process and Testing

The project was built incrementally in phases, each ending with a working application, automated tests and a Git commit (Table 5.3). Design changes from the initial plan were made in response to measured problems, as described above.

Table 5.3 Development phases

Phase	Deliverable
0. Setup	Virtual environment, configuration, LLM wrapper with fallback, SQLite schema, username sign-in
1. Notes Q&A	Extraction, chunking, embeddings, ChromaDB, cross-encoder re-ranking, cited answers, topic extraction
2. Quiz	Topic-based MCQ generation, validation, shuffling, answer-key check for local models, grading, saving answers
3. Weak-topic memory	Topic scores over recent answers, weak/strong classification, adaptive mode, progress dashboard
4. Study plan	Rule-based weighted plan with spaced reviews, mock examination and revision days, regeneration
5. Evaluation	RAG vs no-RAG on 60 examination questions with a blind LLM judge; adaptive-quiz simulation
UI redesign	Custom theme, navigation, dashboard components, one-question-at-a-time quiz, timeline plan

Testing combined four levels: unit tests for pure logic (chunking, header removal, answer and citation parsing, quiz validation, allocation, classification, plan rules and the retry/fallback policy with stubbed models); integration tests with the real embedding and re-ranking models, a temporary ChromaDB and SQLite database and a stubbed LLM; user-interface tests with Streamlit's AppTest, which runs each page headlessly and drives its widgets; and live checks against the real models on the course PDFs. Table 5.4 summarises the automated suite, Table 5.5 gives sample test cases, and Table 5.6 lists defects that were found and fixed.

Table 5.4 Automated test suite (101 tests, all passing)

Test file	Tests	What is covered
test_documents.py	8	Chunk sizes, overlap, page boundaries, header removal, DOCX/TXT extraction, bad files
test_rag.py	5	NOT_FOUND handling, [S#] citation parsing, prompt format, topic JSON parsing
test_ingest.py	4	Ingest → search → answer → delete with real embeddings; re-upload; user isolation
test_llm.py	9	Fallback order, retries, daily quota cool-down, per-minute rate-limit waits
test_quiz.py	14	Validation rules, answer formats, retries for missing questions, answer-key check
test_memory.py	9	Classification, recent-answer window, back-fill, adaptive allocation, topic matching
test_planner.py	10	Coverage of all topics, weighting, spacing, final days, errors, regeneration
test_db.py	5	Schema, users, topic editing keeps ids, cascading deletes
test_eval.py	6	Blind judging order, abstentions, quota stop, summaries, simulation
test_app.py	10	Sign-in, pages, one-question quiz flow, adaptive targeting, progress, study plan

Table 5.5 Sample test cases

ID	Test case	Input / action	Expected result	Result
TC1	Sign in	Enter a username, click Continue	Home page with “Welcome back, <name>”	Pass
TC2	Empty username	Click Continue with no name	Error “Please enter a username.”	Pass
TC3	Unsupported file	Upload a .pptx or an empty .txt	Clear error, nothing stored	Pass
TC4	Cited answer	“What is bucketing in Hive?”	Answer with file + page badges	Pass
TC5	Off-syllabus question	“Who won the 2018 FIFA World Cup?”	“Not found in your notes”, no LLM call	Pass
TC6	Invalid MCQ	Model returns 3 options / “All of the above”	Question dropped and regenerated	Pass
TC7	Incomplete quiz	Submit with unanswered questions	Warning listing missing numbers	Pass
TC8	Answer kept	Answer Q1, Next, Previous	Q1 still shows the chosen option	Pass
TC9	Weak topic	3 answers on a topic, 1 correct	Topic flagged weak (33%)	Pass
TC10	Improvement	10 wrong then 9 right	Topic becomes strong (90% recent)	Pass
TC11	Adaptive quiz	10 questions, one weak topic	6 questions on the weak topic	Pass
TC12	Plan coverage	14-day plan, 10 topics	Every topic studied; weak > untested > strong	Pass
TC13	Plan update	Improve a topic, regenerate	That topic’s planned hours decrease	Pass
TC14	Rate limit	HTTP 429 with retry delay 6 s	Wait and retry the same model	Pass

Table 5.6 Defects found during testing and their fixes

Defect	How it was found	Fix
“What is Apache Pig?” answered “not found”	Live Q&A on course PDFs	Cross-encoder re-ranking of 20 candidates
Header removal deleted “Answer:” / “Example:”	Inspecting extracted text	Only remove repeats at page edges
Model cited “[22]” (a question number)	Live Q&A	Excerpt tags changed to [S1]…[S5]
Local-model quizzes had wrong answer keys	Manual quiz review	Answer-key self-check for Ollama
Quiz page crashed (form key equal to a state key)	AppTest UI test	Renamed the session-state key
Quiz navigator lost clicks after answering	AppTest trace	Widget callbacks update state first
Per-minute limit triggered slow local fallback	Evaluation run stopped at question 16	Wait out short 429 delays on the same model
Re-upload deleted topics and quiz history	Design review in Phase 3	Re-upload updates the document in place

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Evaluation Metrics

The task is open-ended question answering, so classification metrics such as precision, recall and F1 over classes do not apply directly. Each answer is graded against the reference answer as correct, partial or wrong; a "not found" reply counts as wrong. For N questions with C correct, P partial and A abstentions:

$$Accuracy = C / N \tag{6.1}$$

$$Score = (C + 0.5 \cdot P) / N \tag{6.2}$$

$$Answered\ accuracy = C / (N - A) \tag{6.3}$$

Accuracy (Equation (6.1)) is the headline metric; the partial-credit score (Equation (6.2)) and the accuracy among questions the system chose to answer (Equation (6.3)) separate wrong answers from abstentions. For the quiz simulation, the metrics are the share of questions placed on genuinely weak topics, the share of weak topics detected, and the mean mastery of weak topics after ten quizzes, where answering a question on topic t (with its explanation) is assumed to raise its mastery m_t by

$$m_t \leftarrow m_t + 0.08 \cdot (1 - m_t) \tag{6.4}$$

Response times of the main operations are also reported, because the free-tier API and the local model make latency a practical concern.

6.2 Training and Validation Behaviour

Because no model was trained, there are no training or validation loss and accuracy curves. The closest equivalent is to compare the development set, which was used to find problems and try changes, with the held-out set, which was run once on the final pipeline. If design choices had been over-fitted to the development questions, the held-out accuracy would be much lower. Fig. 6.1 shows the development-set results: RAG accuracy was 87% on the development set and 83% on the held-out set (Table 6.1), a one-question difference that is within the judge's noise of about ±2 questions (Section 6.5), so there is no sign of over-fitting.

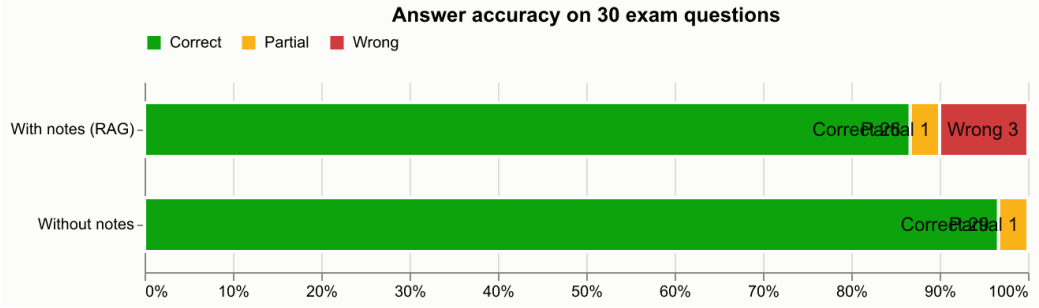


Fig. 6.1 Development set: grading of answers with and without the notes

6.3 Test-Set (Held-Out) Results

Table 6.1 gives the results for both halves; the held-out rows are the test-set result. Fig. 6.2 shows the held-out grading.

Table 6.1 Answer accuracy with and without the notes (held-out rows are the test result)

Set	Condition	Correct	Partial	Wrong	Accuracy	Score
Held-out (30)	With notes (RAG)	25	5	0	83%	92%
	Without notes (B1)	30	0	0	100%	100%
Development (30)	With notes (RAG)	26	1	3	87%	88%
	Without notes (B1)	29	1	0	97%	98%
All 60	With notes (RAG)	51	6	3	85%	90%
	Without notes (B1)	59	1	0	98%	99%

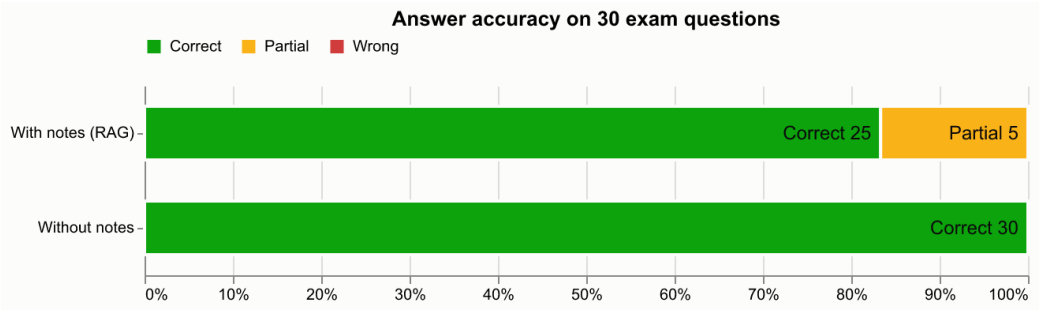


Fig. 6.2 Held-out (test) set: grading of answers with and without the notes

Table 6.2 cross-tabulates the two conditions question by question, in the manner of a confusion matrix. On the held-out set, every question that RAG answered fully correctly was also answered correctly without notes, and the five partial RAG answers were all correct without notes.

Table 6.2 Cross-tabulation of RAG verdict against no-RAG verdict (number of questions)

RAG verdict ↓ / no-RAG verdict →	Held-out: correct	Held-out: partial	Dev: correct	Dev: partial
Correct	25	0	25	1
Partial	5	0	1	0
Wrong (“not found”)	0	0	3	0

Table 6.3 breaks the results down by unit and Table 6.4 compares the answers themselves. RAG answers were about 40% shorter and cited 2.1–2.4 excerpts on average, which makes them quick to check against the notes.

Table 6.3 Accuracy by unit

Set	Unit	Questions	RAG accuracy	No-RAG accuracy
Held-out	III (Pig, Flume)	15	93%	100%
Held-out	IV (Hive, Sqoop)	15	73%	100%
Development	III (Pig, Flume)	15	87%	93%
Development	IV (Hive, Sqoop)	15	87%	100%

Table 6.4 Answer length and citations

Set	Median words (RAG)	Median words (no-RAG)	Mean distinct excerpts cited per RAG answer
Held-out	85	137	2.4
Development (answered questions)	81	138	2.1

Finally, Table 6.5 reports the response times observed on the development machine.

Table 6.5 Observed response times on the development machine

Operation	Observed time
Answer a question (Gemini Flash-Lite)	about 2–7 s
Generate a 10-question quiz (Gemini)	about 15–16 s
Generate a 5-question quiz (local llama3.1:8b, with answer-key check)	about 97–100 s
First local-model reply after start-up (model loading)	about 60 s
Load embedding + re-ranking models (first use after server start)	about 30 s
Embed and index the three course PDFs (67 chunks)	about 26 s

6.4 Comparison with Baselines and Published Work

6.4.1 RAG against the LLM without notes (B1)

On these questions the plain model (B1) was more accurate than the notes-grounded answers: 98% against 85% over all 60 questions. This is the opposite of the expected result, and the reasons matter more than the headline:

- **The notes-grounded answers were never factually wrong.** All three “wrong” verdicts were cases where the system replied “not found in your notes”. When it answered, 96% (development) and 83% (held-out) of its answers were fully correct and the rest were partial.

- **The questions are general knowledge for the model.** Questions such as "What is Apache Hive?" are well covered in the model's training data, so the plain answers are complete, whereas RAG can only use what the retrieved excerpts contain.
- **Where the course has specific points, RAG did better.** For "What are the benefits of using schemas?", the RAG answer gave the course's points (fields referenced by name) and was graded correct, while the plain answer gave generic database benefits and was graded partial.

This evaluation does not measure two benefits of grounding: answers that follow the course's own terminology and emphasis, and the ability to verify every claim against a cited page. A question set built from material that is *not* general knowledge, such as lecturer-specific examples, would be a fairer test of that value.

6.4.2 Adaptive against even-mix quizzes (B4)

A real comparison would require many students over several weeks, so the adaptive strategy was evaluated by simulation using the application's own allocation and scoring code (setup in Table 4.1, learning assumption in Equation (6.4)). Table 6.6 gives the results and Fig. 6.3 shows weak-topic mastery per quiz.

Table 6.6 Simulation results: adaptive versus even-mix quizzes (200 simulated students per mode)

Measure	Adaptive	Even mix (B4)
Questions on genuinely weak topics (quizzes 4–10)	55%	25%
Weak topics flagged as weak at least once in 10 quizzes	73%	70%
Students whose weak topics were all flagged within 10 quizzes	18%	10%
Quizzes needed to flag all weak topics (mean, when flagged)	6.4	8.5
Mean mastery of weak topics after 10 quizzes (start ≈ 35%)	68%	55%

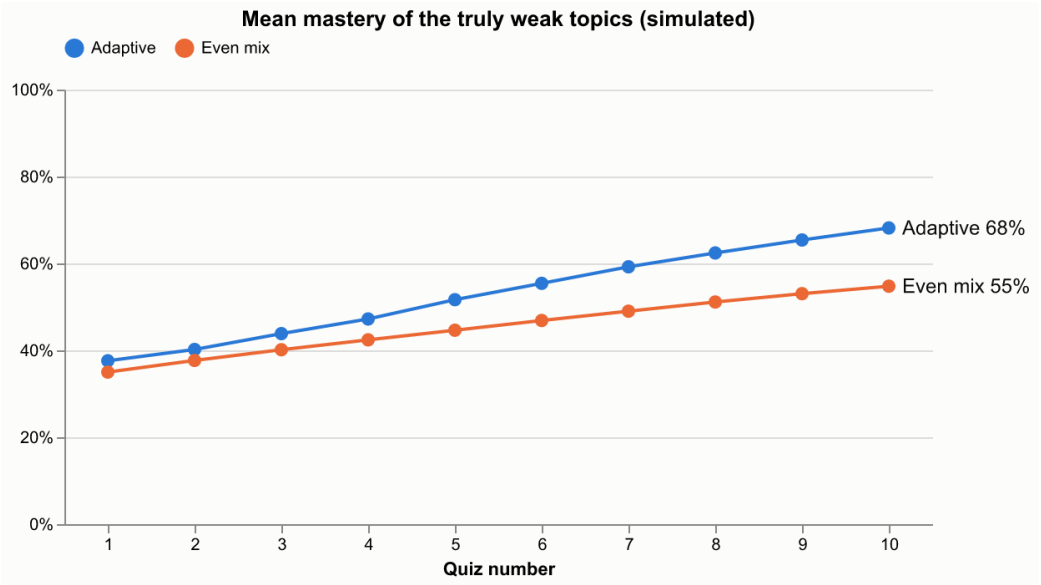


Fig. 6.3 Simulated mastery of the genuinely weak topics per quiz

Adaptive mode spent more than twice as much practice on the weak topics once they were identified and, under the stated assumption, raised their mastery by about 13 points more than an even mix. It did not identify weak topics much faster: because a topic is judged after only three answers, a weak topic with about 40% accuracy still scores 2 of 3 by chance about a third of the time. The size of the mastery gain depends on the assumed learning rate; the share of questions on weak topics does not.

6.4.3 Relation to published work

The numbers above cannot be compared directly with published benchmarks, because those use different data and tasks. Qualitatively, the findings are consistent with the literature. RAG's advantage reported by Lewis et al. [3] is largest for knowledge the model does not already have; on general-knowledge course questions the plain model is a strong baseline. The failure analysis below agrees with the RAG survey [8] that chunking and retrieval quality, rather than generation, dominate end-to-end accuracy. The re-ranker's large effect on the "Apache Pig" question mirrors the gains from cross-encoder re-ranking reported by Nogueira and Cho [14].

6.5 Error Analysis

Table 6.7 lists every notes-grounded answer that was not fully correct, together with its cause, found by inspecting the retrieved chunks and their ranks.

Table 6.7 Analysis of notes-grounded answers that were not fully correct

Question	Verdict	Cause
What is a Map in Pig?	Not found	The defining bullet ("Map: collection of key-value pairs…") never mentions "Pig"; "Map" matched MapReduce chunks, ranking it 16th
What are conditional functions in Hive?	Not found	The IF/CASE chunk was not among the 20 embedding candidates
What is a Subquery in Hive?	Not found	A chunk that only mentions subqueries ranked above the defining summary table
How can data be imported into Hive?	Partial	"Import" retrieved the Sqoop import pages instead of Hive's LOAD DATA section
What is Bucketing in Hive?	Partial	The chunk boundary cut the CREATE TABLE … CLUSTERED BY example in half
What are UDFs / relational operators in Hive?	Partial	Retrieved excerpts listed fewer items (no UDAF/UDTF; no LIKE/RLIKE/IN) than the reference
What is a Tuple in Pig? / JOIN in Pig?	Partial	Answer missed points (ordered fields; common key) present in the reference but not in the retrieved text

The common cause is **chunks that lose their context**: a 180-word chunk from the middle of a section does not contain the section heading ("Pig Latin – Data Types", "Hive – Importing Data") that would connect it to the question. Unit IV (Hive, Sqoop) was harder (73% held-out accuracy against 93% for Unit III) because Hive and Sqoop share vocabulary such as "import" and "table". Fig. 6.6 in Section 6.7 shows a visible example: the SQL example in an answer is cut off after `CREATE TABLE student (` because the chunk containing it ends there.

Reliability of the judge. In the BM25 experiment the unchanged no-notes answers were re-judged and scored 27 correct instead of 29, so the judge varies by about ±2 questions in 30; differences of 1–2 questions between runs are not meaningful. The RAG–no-RAG gap over 60 questions (51 against 59 correct) is larger than this, so its direction is reliable for this question set. The judge is the same model that wrote the answers; grading against a written reference and blind ordering limit self-preference [19], and a manual check of the verdicts (columns are provided in the results files) would strengthen the result.

6.6 Ablation Study

Table 6.8 summarises the effect of removing or changing components. Only the first two rows are controlled measurements on the full development set; the others were observed on specific cases during development and are reported as such.

Table 6.8 Ablation study

Variant	Evidence	Result	Decision
Add BM25 keyword search to stage 1 (B2)	Development set, end to end	RAG 25 vs 26 correct (no gain); missing chunks reached the candidates but were still ranked low	Reverted
Larger re-ranker MiniLM-L-12 (B3)	Development set, retrieval only (chunk covering ≥ 50% of the reference reaches the LLM)	28/30 vs 27/30	Not adopted (cost vs one question)
Remove the cross-encoder	"What is Apache Pig?"	Defining chunk ranked 8th → answer "not found"; with re-ranker ranked 1st	Kept re-ranker
500-token chunks instead of 180 words	Model specification	Text beyond 256 word-pieces silently ignored by the embedding model	180 / 40 words
Header removal over the whole page	Inspection of extracted text	Deleted "Answer:"/"Example:" labels in the body	Edges only
Remove the answer-key check (local model)	Manual review of 5 questions	2 of 5 questions had wrong keys	Check kept for Ollama
All-time accuracy instead of last 10 (B5)	Worked example / unit test	10 wrong then 9 right: 47% (weak) vs 90% (strong)	Last 10 answers
Numeric tags [1] instead of [S1]	Live Q&A	Model cited a question number from the notes as a source	[S#] tags

6.7 Output Screenshots

The screenshots below were captured from the running application. The Notes Q&A answers and quiz questions are real outputs generated from the Big Data course notes. Screens that depend on quiz history (Home, Progress and the Study Plan timeline) use a sample quiz history in a demonstration copy of the database, because the student account had only one quiz at the time of capture. Fig. 6.4 to Fig. 6.14 show each page in turn.

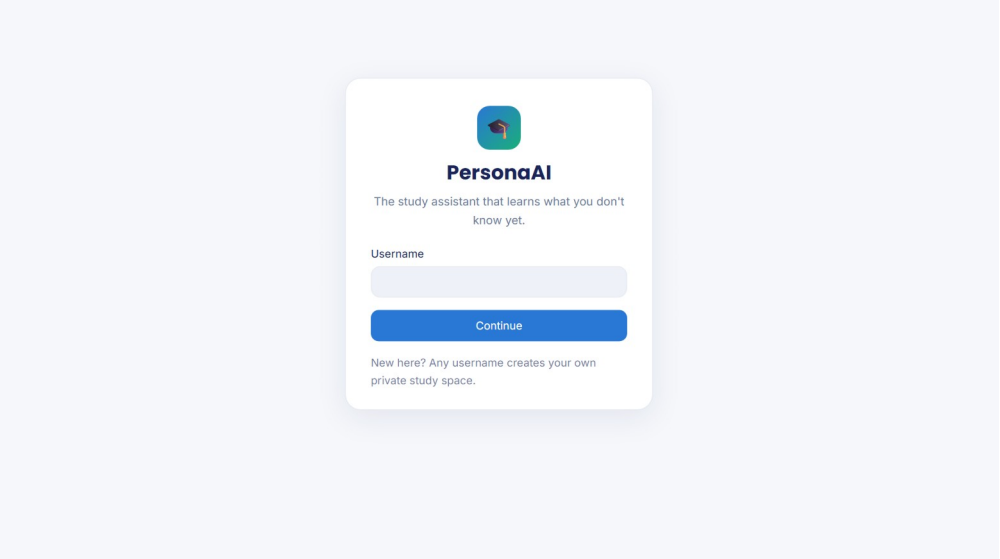


Fig. 6.4 Sign-in page

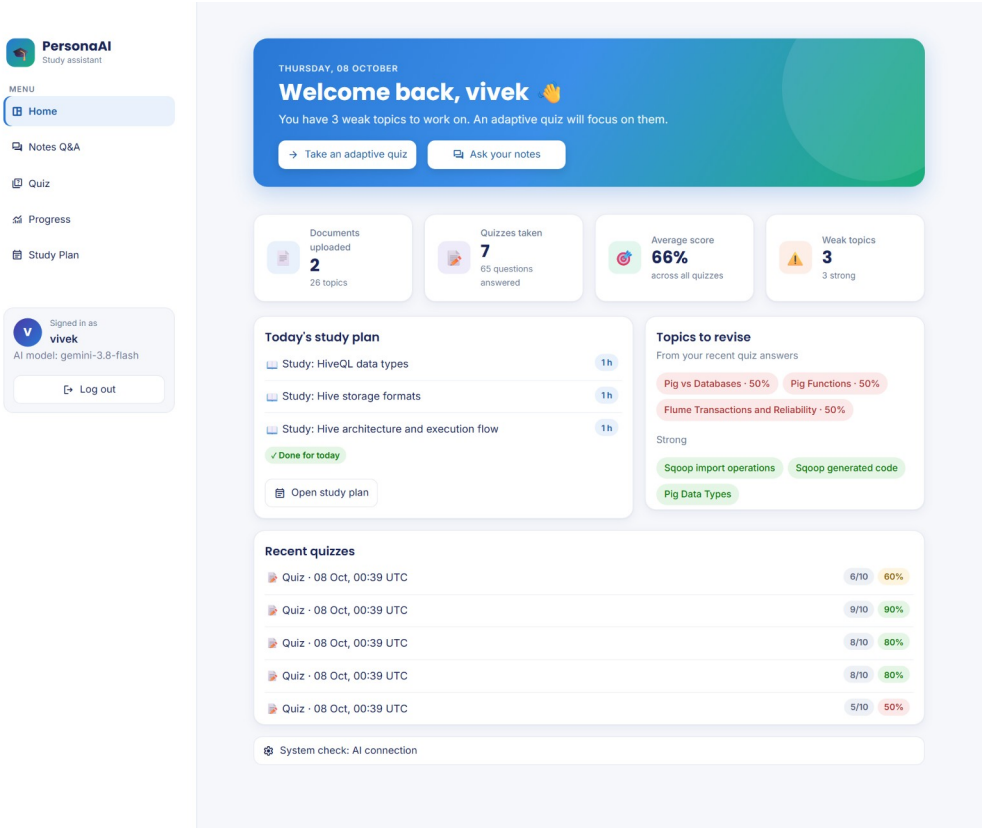


Fig. 6.5 Home dashboard: welcome banner, statistics, today's plan and topics to revise (sample quiz history)

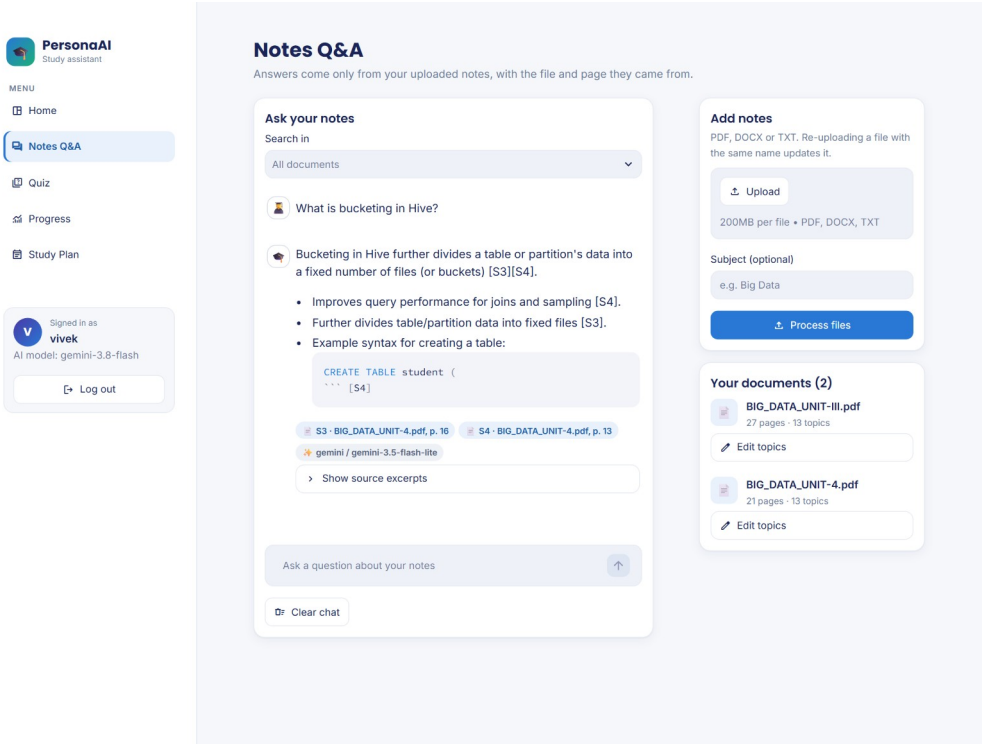


Fig. 6.6 Notes Q&A: a real answer from the notes with file and page citations

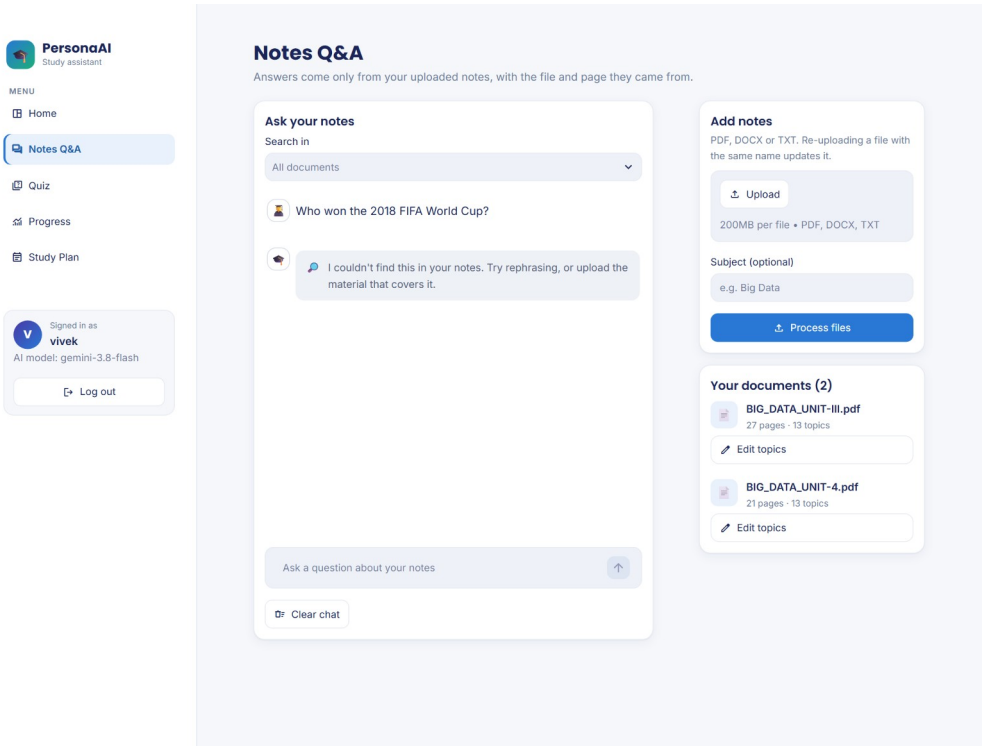


Fig. 6.7 Notes Q&A: an off-syllabus question is refused instead of answered from general knowledge

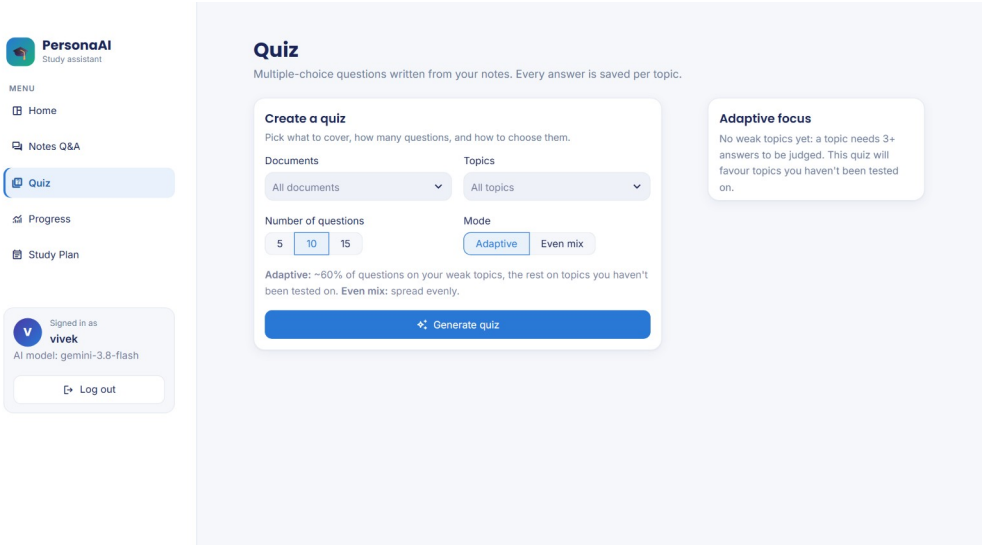


Fig. 6.8 Quiz setup: documents, topics, number of questions and mode

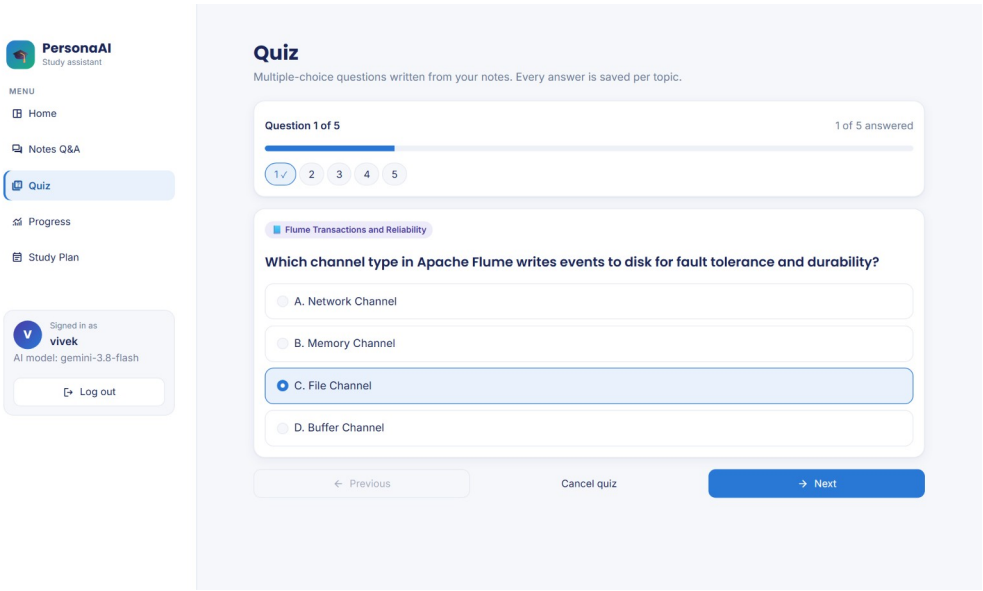


Fig. 6.9 Taking a quiz: one question at a time with progress and question navigator (real generated question)

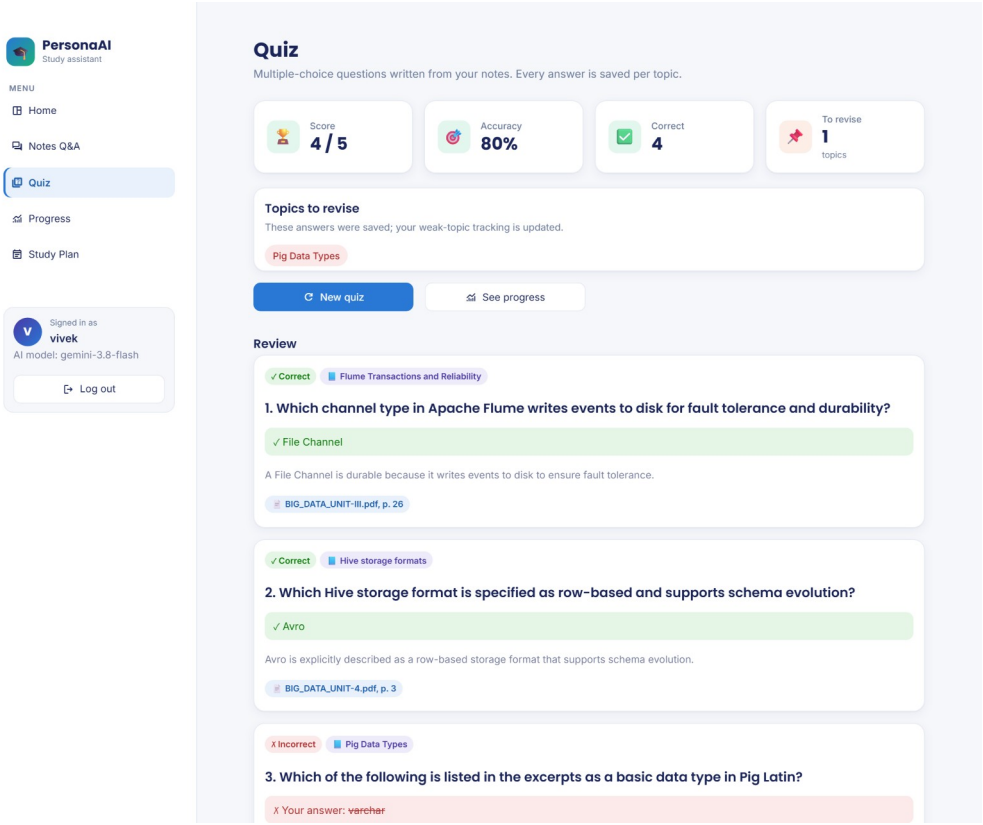


Fig. 6.10 Quiz results with per-question review, explanations and sources

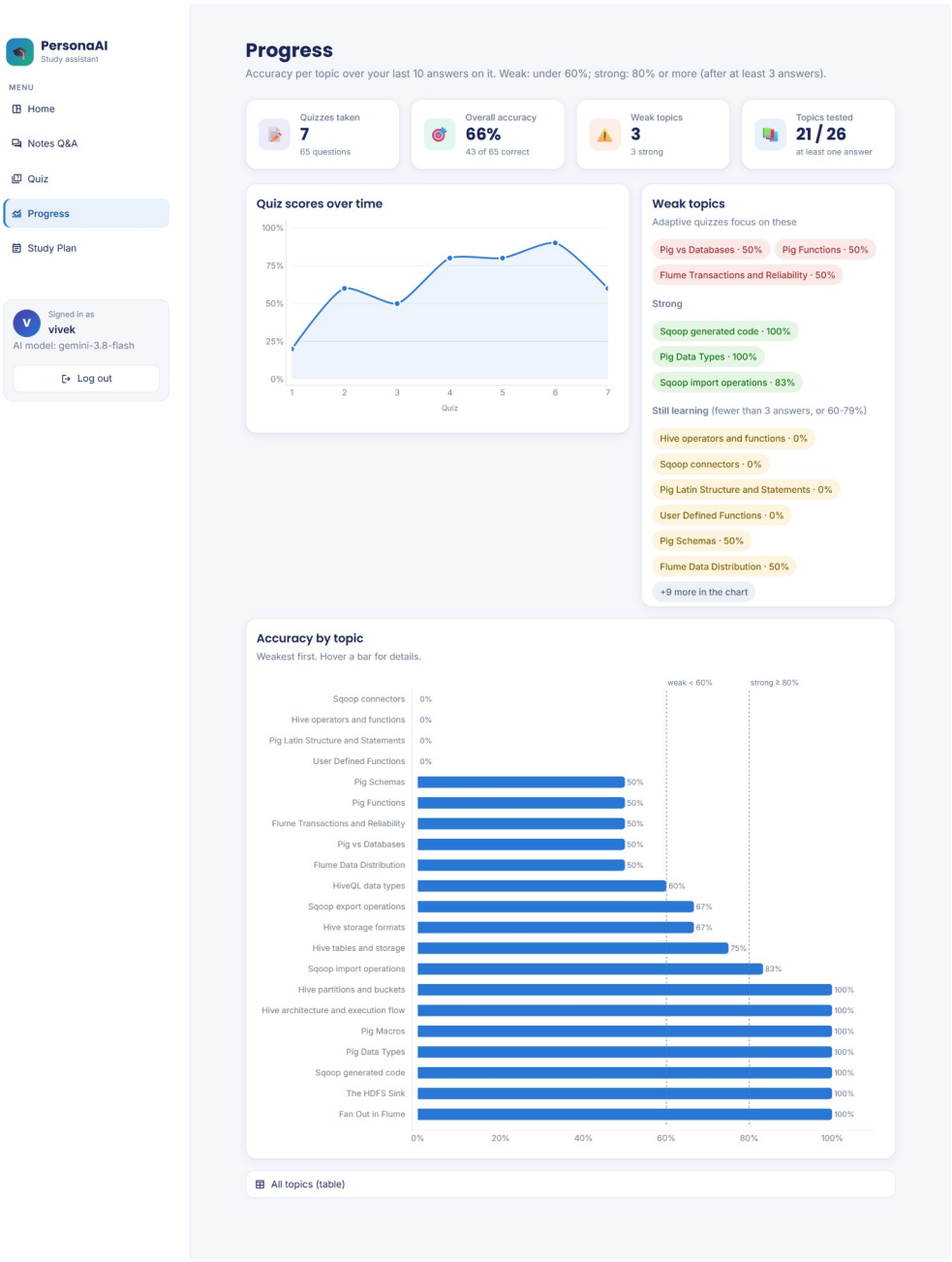


Fig. 6.11 Progress dashboard: score trend, weak and strong topics, accuracy per topic (sample quiz history)

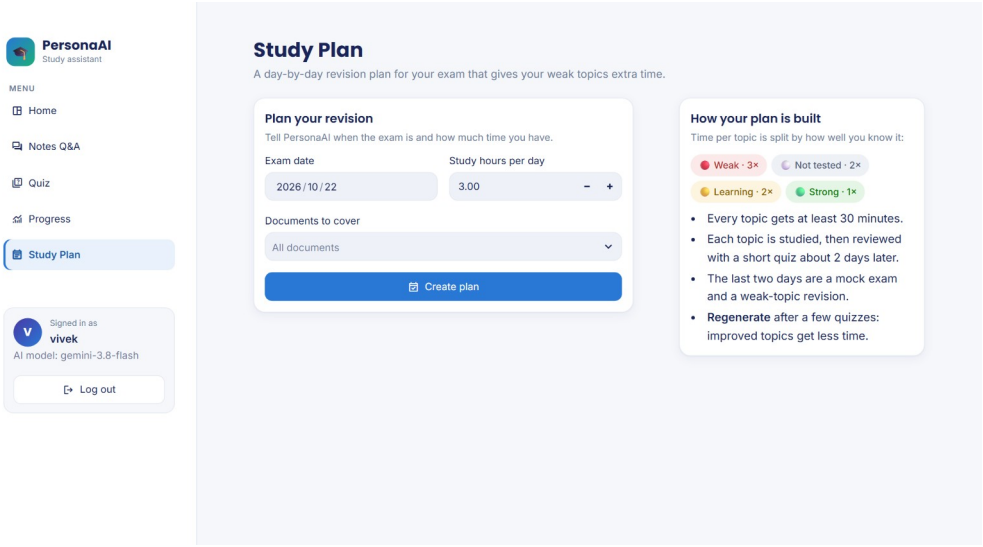


Fig. 6.12 Creating a study plan

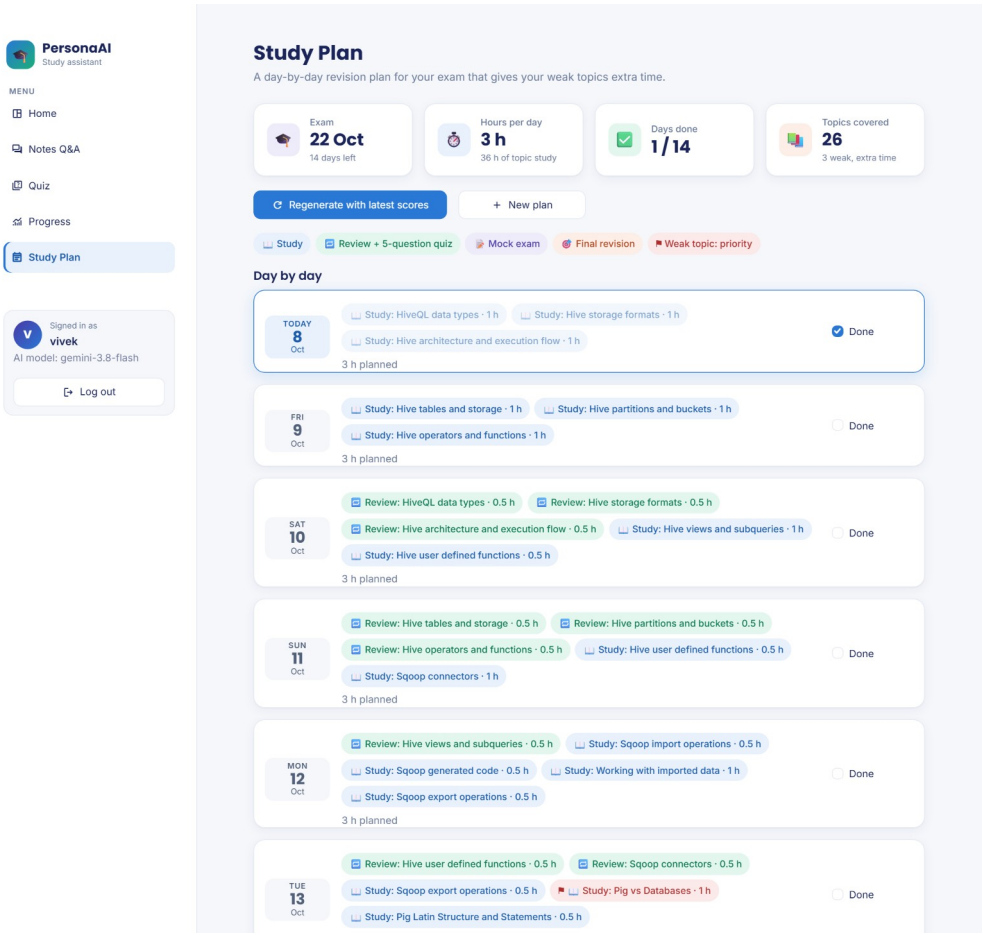


Fig. 6.13 Study plan timeline with task chips and weak-topic priority flags (sample quiz history)

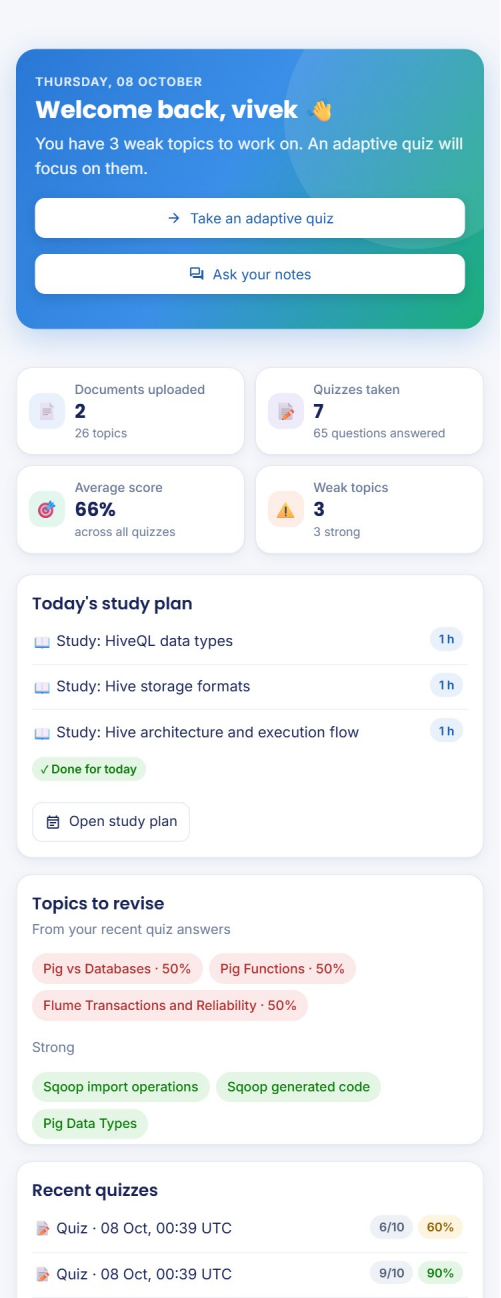


Fig. 6.14 Responsive layout on a phone-sized screen

6.8 Ethical and Responsible AI

- **Data privacy.** Uploaded notes are stored locally (`data/`, git-ignored), and every query is filtered by the student's user id. In online mode, the retrieved excerpts and the question are sent to the Gemini API, a third-party service, so students should not upload personal or confidential documents; offline mode keeps all processing on the computer. The API key is kept out of the code. Username-only sign-in means anyone who knows a username can see that space, which is acceptable for a classroom demonstration but not for public deployment.

- **Hallucination and over-reliance.** Grounding, inline citations and the "not found" reply reduce, but do not eliminate, the risk of wrong answers [2]. Showing the cited excerpts encourages students to check answers rather than trust them blindly, in line with the concerns raised by Kasneci et al. [1].
- **Bias and fairness.** The evaluation covers one course in English and 60 questions; results may not transfer to other subjects or languages. The LLM judge may share biases with the answering model [19]. The learner model treats all students the same and uses fixed thresholds, which may be harsh for topics that are inherently harder.
- **Limitations of the evidence.** The adaptive-quiz benefit is a simulation result under an assumed learning rate, not a measured improvement in real students' marks, and is reported as such.
- **Misuse risks.** The tool could be used to generate quiz questions from copyrighted material that the user has no right to share, or during closed-book assessments. It is intended for personal revision of course material the student legitimately has; it does not publish or share uploaded content.
- **Accessibility and environmental cost.** Status is never conveyed by colour alone, and the interface works on phones. Small local models for embedding and re-ranking keep the compute and energy cost of each query low.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

PersonaAI shows that a study assistant can be grounded in a student's own course material, remember each student's weak topics, and use that memory to direct both practice and planning. Against the objectives of Section 1.4:

- Notes in PDF, DOCX and TXT are indexed per student (48 chunks for the 48 pages of the course notes) and answered with file-and-page citations, about 2.4 excerpts cited per answer on the held-out set.
- Off-syllabus questions are refused: no notes-grounded answer in the 60-question evaluation was factually wrong; the three failures were "not found" replies.
- Quizzes of 5, 10 or 15 validated MCQs are generated in about 15 s with Gemini, with option shuffling and an answer-key check for the local model.
- Weak-topic memory, the adaptive mode and the study planner work as specified; in simulation, adaptive quizzes placed 55% of questions on genuinely weak topics against 25% for an even mix and raised weak-topic mastery from about 35% to 68% against 55%.
- The evaluation was honest about its result: on general-knowledge examination questions, grounded answers scored 83% on the held-out set against 100% for the same model without notes, with retrieval of the right passage identified as the cause.
- The application keeps working when the online API is rate-limited, through the lighter model and the offline Llama 3.1 fallback, and is covered by 101 automated tests.

7.2 Limitations

- Retrieval misses or truncates some answers because fixed-size chunks lose their section headings.
- The evaluation uses a single course, 60 questions and an LLM judge without a completed manual check of verdicts.

- The adaptive-quiz benefit was measured by simulation rather than with real students.
- Weak-topic detection after three answers is noisy.
- The free-tier API is rate-limited; the local model is slower and writes lower-quality quiz questions.
- Sign-in is by username only, without passwords; scanned (image-only) PDFs are not supported.

7.3 Future Enhancements

- Prefix each chunk with its section heading and split on section boundaries, the main fix suggested by the error analysis.
- Evaluate stronger embedding models and re-rankers (Section 4.8) on the same development and held-out sets.
- Offer an optional, clearly labelled “notes first, then general knowledge” answer mode.
- Replace the threshold learner model with Bayesian Knowledge Tracing or half-life regression once enough real answers are available, and run a study with real students over a semester.
- Add password or institutional sign-in, a hosted database, and a teacher view of class-wide weak topics.
- Add OCR for scanned notes and support for lecture slides and handwritten notes.

REFERENCES

[1] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, art. no. 102274, 2023.

[2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, art. no. 248, pp. 1–38, 2023.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[4] H. L. Roediger and J. D. Karpicke, "Test-enhanced learning: Taking memory tests improves long-term retention," *Psychological Science*, vol. 17, no. 3, pp. 249–255, 2006.

[5] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, and D. Rohrer, "Distributed practice in verbal recall tasks: A review and quantitative synthesis," *Psychological Bulletin*, vol. 132, no. 3, pp. 354–380, 2006.

[6] J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham, "Improving students' learning with effective learning techniques: Promising directions from cognitive and educational psychology," *Psychological Science in the Public Interest*, vol. 14, no. 1, pp. 4–58, 2013.

[7] United Nations, "Goal 4: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all," Sustainable Development Goals. [Online]. Available: https://sdgs.un.org/goals/goal4 (accessed Oct. 26, 2026).

[8] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[9] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[10] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[11] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. Conf. Empirical Methods in Natural Language Processing and Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992.

[13] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[14] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” arXiv:1901.04085, 2019.

[15] P. Bajaj et al., “MS MARCO: A human generated machine reading comprehension dataset,” arXiv:1611.09268, 2016.

[16] L. Wang et al., “Text embeddings by weakly-supervised contrastive pre-training,” arXiv:2212.03533, 2022.

[17] S. Xiao et al., “C-Pack: Packed resources for general Chinese embeddings,” in *Proc. 47th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2024.

[18] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Computational Linguistics*, vol. 12, pp. 157–173, 2024, doi: 10.1162/tacl_a_00638.

[19] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[20] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAs: Automated evaluation of retrieval augmented generation,” in *Proc. 18th Conf. European Chapter of the Assoc. Computational Linguistics: System Demonstrations*, St. Julian’s, Malta, 2024, pp. 150–158.

[21] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, “A systematic review of automatic question generation for educational purposes,” *Int. J. Artificial Intelligence in Education*, vol. 30, no. 1, pp. 121–204, 2020, doi: 10.1007/s40593-019-00186-y.

[22] A. T. Corbett and J. R. Anderson, “Knowledge tracing: Modeling the acquisition of procedural knowledge,” *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.

[23] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, “Deep knowledge tracing,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 505–513.

[24] B. Settles and B. Meeder, “A trainable spaced repetition model for language learning,” in *Proc. 54th Annu. Meeting of the Assoc. Computational Linguistics (ACL)*, Berlin, Germany, 2016, pp. 1848–1858, doi: 10.18653/v1/P16-1174.

[25] D. Wu et al., “LongMemEval: Benchmarking chat assistants on long-term interactive memory,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[26] Snowflake Inc., “Streamlit documentation,” 2026. [Online]. Available: https://docs.streamlit.io (accessed Oct. 26, 2026).

[27] Chroma, “Chroma documentation,” 2026. [Online]. Available: https://docs.trychroma.com (accessed Oct. 26, 2026).

[28] N. Reimers et al., “Sentence Transformers documentation,” 2026. [Online]. Available: https://www.sbert.net (accessed Oct. 26, 2026).

[29] Google, “Gemini API documentation,” 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs (accessed Oct. 26, 2026).

[30] Ollama, “Ollama: Get up and running with large language models,” 2026. [Online]. Available: https://ollama.com (accessed Oct. 26, 2026).

[31] Gemini Team, Google, “Gemini: A family of highly capable multimodal models,” arXiv:2312.11805, 2023.

[32] Llama Team, AI @ Meta, “The Llama 3 herd of models,” arXiv:2407.21783, 2024.

APPENDICES

Appendix A: GitHub Repository and README

Repository: [GitHub repository URL — add after pushing the code]

The README describes the problem, features, tech stack and architecture, and gives the commands below to set up the project and reproduce every result in Chapter 6.

Listing A.1 Setup and reproduction commands

```
git clone <repository URL>
cd PersonaAI
py -3.12 -m venv .venv
.venv\Scripts\activate     # Linux/macOS: source .venv/bin/activate
pip install -r requirements-dev.txt
# .env: GEMINI_API_KEY=..., LLM_PROVIDER=gemini
streamlit run app.py

pytest                                  # 101 tests
python -m evaluation.build_questions  # question sets
python -m evaluation.run_eval --out evaluation/results/dev_v1
python -m evaluation.run_eval \
    --questions evaluation/questions_holdout.csv \
    --out evaluation/results/holdout
python -m evaluation.simulate_adaptive --students 200
```

Offline mode: install Ollama, run `ollama pull llama3.1:8b`, and set `LLM_PROVIDER=ollama` in `.env`. The course PDFs are not included in the repository (see the licence note in Table 4.1).

Appendix B: Model Links

No model was trained, so there are no trained weights to publish. The pretrained models used are:

- Embedding model: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- Re-ranker: https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2
- Local LLM: https://ollama.com/library/llama3.1 (tag `llama3.1:8b`)
- Online LLM: Google Gemini API, models `gemini-3.8-flash` and `gemini-3.5-flash-lite`

Appendix C: Project Structure and Key Code

The key code is explained in Section 5.3. The project is organised as follows:

Listing C.1 Project structure

```
PersonaAI/
├── app.py              # sign-in card + page router
├── views/              # home, notes_qa, quiz, progress, study_plan
├── ui/                 # styles.py (theme), shell.py (sidebar)
├── core/               # config, documents, vectorstore, ingest,
│                       # rag, llm, quiz, memory, planner, db, charts
├── evaluation/         # build_questions, run_eval,
│                       # simulate_adaptive, REPORT.md, results/
├── tests/              # 101 tests (pytest + Streamlit AppTest)
├── .streamlit/         # config.toml: theme colours and fonts
├── data/               # uploads, ChromaDB, SQLite (git-ignored)
├── requirements.txt, requirements-dev.txt
├── README.md
└── IMPLEMENTATION.md
```

Table C.1 Modules and their responsibilities

Module	Responsibility	Lines
core/config.py	Settings from .env or Streamlit secrets; paths; logging	63
core/documents.py	PDF/DOCX/TXT extraction, header/footer removal, page-bounded chunking	175
core/vectorstore.py	Local embeddings and ChromaDB storage/search, filtered by user	107
core/ingest.py	Upload pipeline: save, extract, chunk, embed, extract topics	126
core/rag.py	Two-stage retrieval, grounded prompt, citation parsing, “not found”	127
core/llm.py	One interface to Gemini and Ollama: retries, rate limits, fallback	206
core/quiz.py	Allocation, MCQ generation, validation, shuffling, key check, grading	333
core/memory.py	Topic statistics, classification, adaptive allocation, topic matching	138
core/planner.py	Rule-based weighted study plan; save and regenerate	260
core/db.py	SQLite schema and queries; topic scores rebuilt after every quiz	288
core/charts.py	Plotly charts for the Progress page	75
app.py, ui/, views/	Sign-in, router, sidebar, theme and the five pages	1,267
evaluation/	Question builder, RAG vs no-RAG runner, adaptive-quiz simulation	641

Appendix D: Plagiarism Report

[Attach the similarity report here. The similarity index must be below 20%, excluding references.]

Appendix E: Publications and Certificates

[None / list any publications or certificates arising from this project.]